Qualcomm Technologies, Inc.

Audio Playback/Record in AudioReach

80-VN500-26 Rev. AB

June 16, 2021

Qualcomm Confidential - May Contain Trade Secrets
2022-11-05 10:34:02 GMT
nijiasheng@longcheer.com

For additional information or to submit technical questions, go to: https://createpoint.qti.qualcomm.com

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

Confidential Distribution: Use or distribution of this item, in whole or in part, is prohibited except as expressly permitted by written agreement(s) and/or terms with Qualcomm Incorporated and/or its subsidiaries.

Not to be used, copied, reproduced, or modified in whole or in part, nor its contents revealed in any manner to others without the express written permission of Qualcomm Technologies, Inc.

All Qualcomm products mentioned herein are products of Qualcomm Technologies, Inc. and/or its subsidiaries.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Other product and brand names may be trademarks or registered trademarks of their respective owners.

This technical data may be subject to U.S. and international export, re-export, or transfer (“export”) laws. Diversion contrary to U.S. and international law is strictly prohibited.

Qualcomm Technologies, Inc.
5775 Morehouse Drive
San Diego, CA 92121
U.S.A.

© 2021 Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision history

Revision	Date	Description
AA	February 2021	Initial release
AB	June 2021	Updated some images for improved quality

Qualcomm
Confidential - May Contain Trade Secrets
2022-11-05 10:34:02 GMT
nijiasheng@longcheer.com

Contents

1 Introduction 6

1.1 Purpose 6
1.2 Conventions 6
1.3 Technical assistance 6

2 AudioReach architecture 7

2.1 Platform audio layer (PAL) 7
2.2 Resource manager 9
2.3 ALSA plugins 12
2.4 Snd-card-def parser 14
2.5 Audio graph manager (AGM) 14
2.5.1 Session object 15
2.5.2 Graph object 16
2.5.3 Device object 16
2.6 GSL 16
2.6.1 GSL API layer 17
2.6.2 Graph handling 17
2.6.3 Subgraph and concurrency handling 17
2.6.4 Set/get config 17
2.6.5 Buffer management 17
2.6.6 Event processing 17
2.6.7 SPF router 18
2.7 AML 18

3 AudioReach playback use cases 19

4 AudioReach record use cases 22

5 AudioReach debugging 23

5.1 Definitions 23
5.2 Dynamic logging 23
5.3 AGM command line 24
5.4 GSL commands tool 25
5.5 Log analysis 25

6 SPF audio playback 33

6.1 Overview 33
6.1.1 Audio data IN 33
6.1.2 Processing 33

Qualcomm Confidential – May Contain Trade Secrets
2024-11-05 10:34:02 GMT
nujabe@engcollingcheer.com

6.1.3 Audio data OUT ... 33
6.2 Playback capabilities ... 33
6.2.1 Data type based ... 34
6.2.2 Data flow based ... 35
6.2.3 Session control related ... 36
6.3 Framework modules for playback use cases ... 37
6.3.1 WR SH Mem EP ... 37
6.3.2 SH Mem Pull Mode EP ... 38
6.3.3 Placeholder Decoder ... 39
6.3.4 PCM Converter ... 39
6.3.5 Media Format Converter (MFC) ... 40
6.3.6 Splitter Renderer (SPR) ... 41
6.3.7 Splitter ... 42
6.3.8 Simple Accumulator and Limiter (SAL) ... 43
6.3.9 Hardware Endpoint Sink module ... 44
6.3.10 Gapless module ... 45
6.4 Playback operations ... 46
6.4.1 Typical playback graph ... 46
6.4.2 Start/run ... 48
6.4.3 Pause, seek, and resume ... 50
6.4.4 Suspend ... 53
6.4.5 Device switch ... 54

7 SPF audio record ... 56
7.1 Overview ... 56
7.1.1 Audio data IN ... 56
7.1.2 Processing ... 56
7.1.3 Audio data OUT ... 56
7.2 Framework modules for record use cases ... 57
7.2.1 RD SH Mem EP ... 57
7.2.2 SH Mem Push mode ... 57
7.2.3 Generic Synchronization ... 58
7.2.4 Priority Synchronization ... 59
7.2.5 Multiport Sample Slip ... 60
7.2.6 Placeholder Encoder ... 60
7.3 Use cases ... 61
7.3.1 Basic record ... 61
7.3.2 Record with Push Mode EP module ... 63
7.3.3 Loopback ... 63
7.3.4 Karaoke ... 65
7.3.5 Hands-Free Profile (HFP) ... 67

8 SPF debugging ... 69
8.1 Graph-related ... 69
8.1.1 Graph management commands ... 69
8.1.2 Graph config commands ... 70
8.1.3 Media format propagation ... 71
8.2 Module-related ... 71
8.2.1 Hardware EP modules ... 71

Qualcomm
Confidential - May Contain Trade Secrets
2022-12-05 10:34:02 GMT
jiao@angelomacheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.2.2 WR SH Mem EP 72
8.2.3 RD SH Mem EP 73
8.2.4 Data Logging 73
8.2.5 MFC 73
8.2.6 Placeholder 74
8.2.7 SPR 74
8.2.8 Soft Pause 74
8.3 Common issues 75
8.3.1 Output is muted 75
8.3.2 Output is noisy 76

A References 77

A.1 Related documents 77
A.2 Acronyms and terms 77

Qualcomm
Confidential - May Contain Trade Secrets
2022-11-05 10:34:02 GMT
nijiasheng@longcheer.com

1 Introduction

1.1 Purpose

This document provides hardware and software architecture details for the audio playback and record use cases with AudioReach™ and the signal processing framework (SPF).

1.2 Conventions

Function declarations, function names, type declarations, attributes, and code samples appear in a different font, for example, `cp armcc armcpp`.

Code variables appear in angle brackets, for example, `<number>`.

Commands to be entered appear in a different font, for example, **`copy a:*.* b:`**.

Button and key names appear in bold font, for example, click **Save** or press **Enter**.

Shading indicates content that has been added or changed in this revision of the document.

1.3 Technical assistance

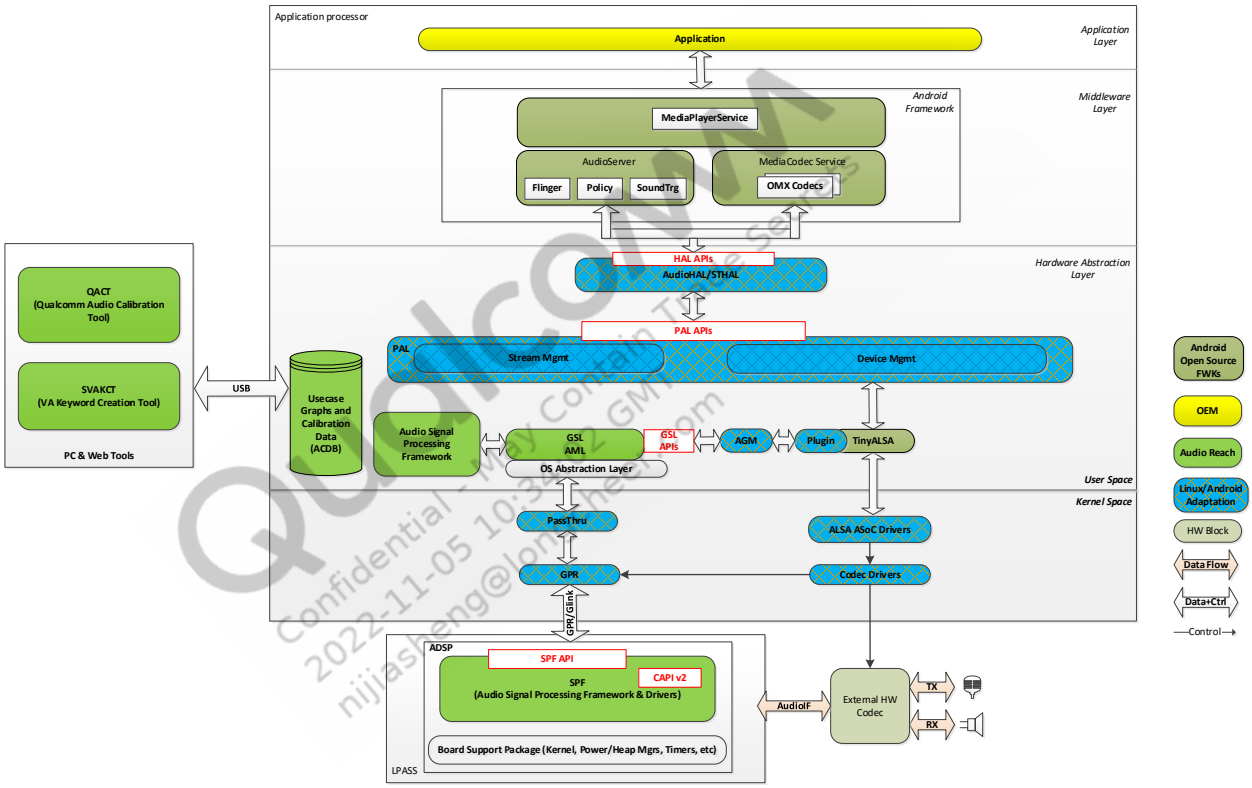
For assistance or clarification on information in this document, submit a case to Qualcomm Technologies, Inc. (QTI) at https://createpoint.qti.qualcomm.com/.

If you do not have access to the CDMATech Support website, register for access or send email to support.cdmatech@qti.qualcomm.com.

Qualcomm Confidential – May Contain Trade Secrets 2022-11-05 10:31:02 GMT xiaosheng@longchuer.com

2 AudioReach architecture

The following figure shows the overall audio software architecture.



2.1 Platform audio layer (PAL)

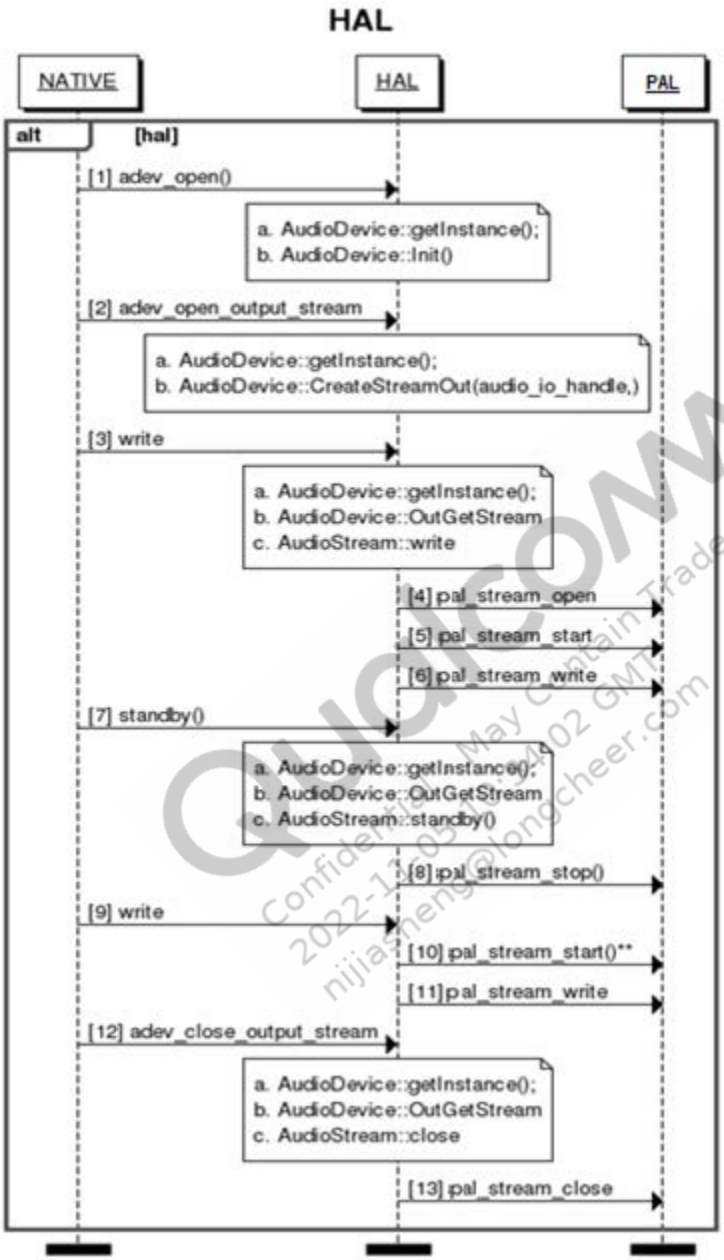
The PAL provides higher-level audio-specific APIs to access the underlying QTI audio hardware and drivers to enable feature rich audio and voice use cases including:

- Audio playback and record in PCM and compressed formats with various QTI pre- and postprocessing modules including latency and power sensitive use cases
- Low power voice activation use cases with various pre- and postprocessing modules
- Voice calls use cases
- Hardware accelerated encoding/decoding or transcoding of audio formats

Clients can interface with PAL using various APIs for controlling and configuring sessions and devices.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The following call flow shows how the HAL interacts with PAL.

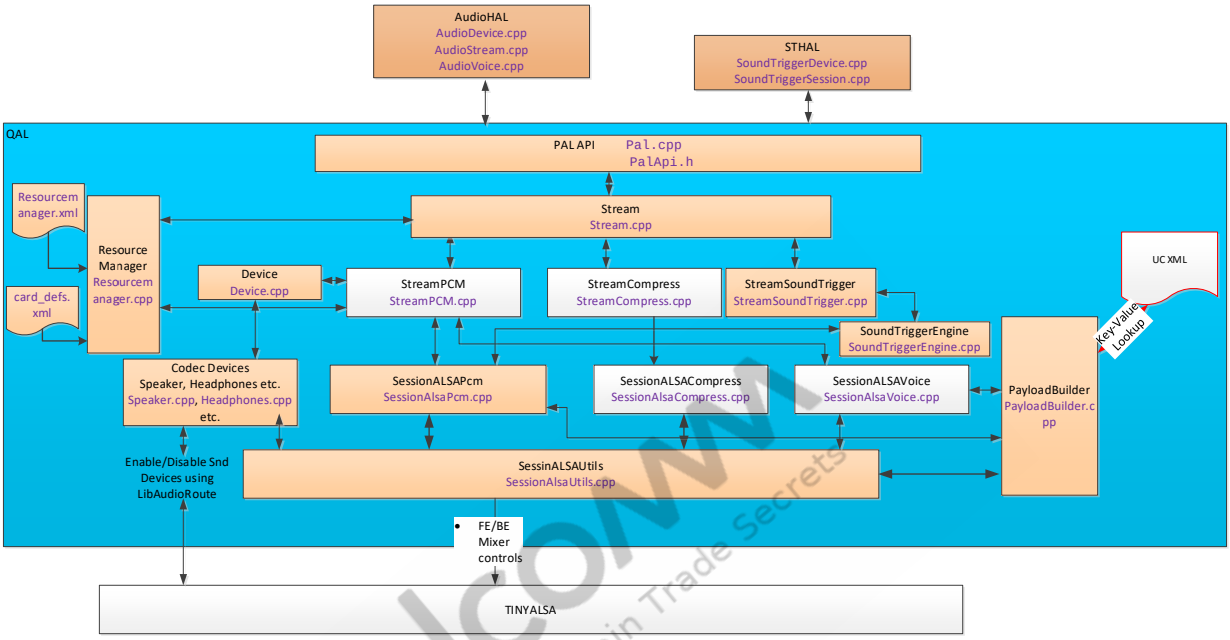


As shown in the call flow, PAL performs the following operations:

- Configures mixer controls to set up the following:
  - Front-end and back-end connections
  - Set up graph key vector (GKVs) and calibration key vectors (CKVs)
  - Codec blocks
  - Module configuration (if required for the use case)
- Invokes TinyALSA APIs to open/start audio sessions using PCM and compress ALSA user space APIs

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The following diagram shows the blocks involved in PAL.



2.2 Resource manager

ResourceManager is a singleton class – Only one instance of resource manager is needed at any time. Resource manager loading is done in pal_init(). Resource manager performs the following operations:

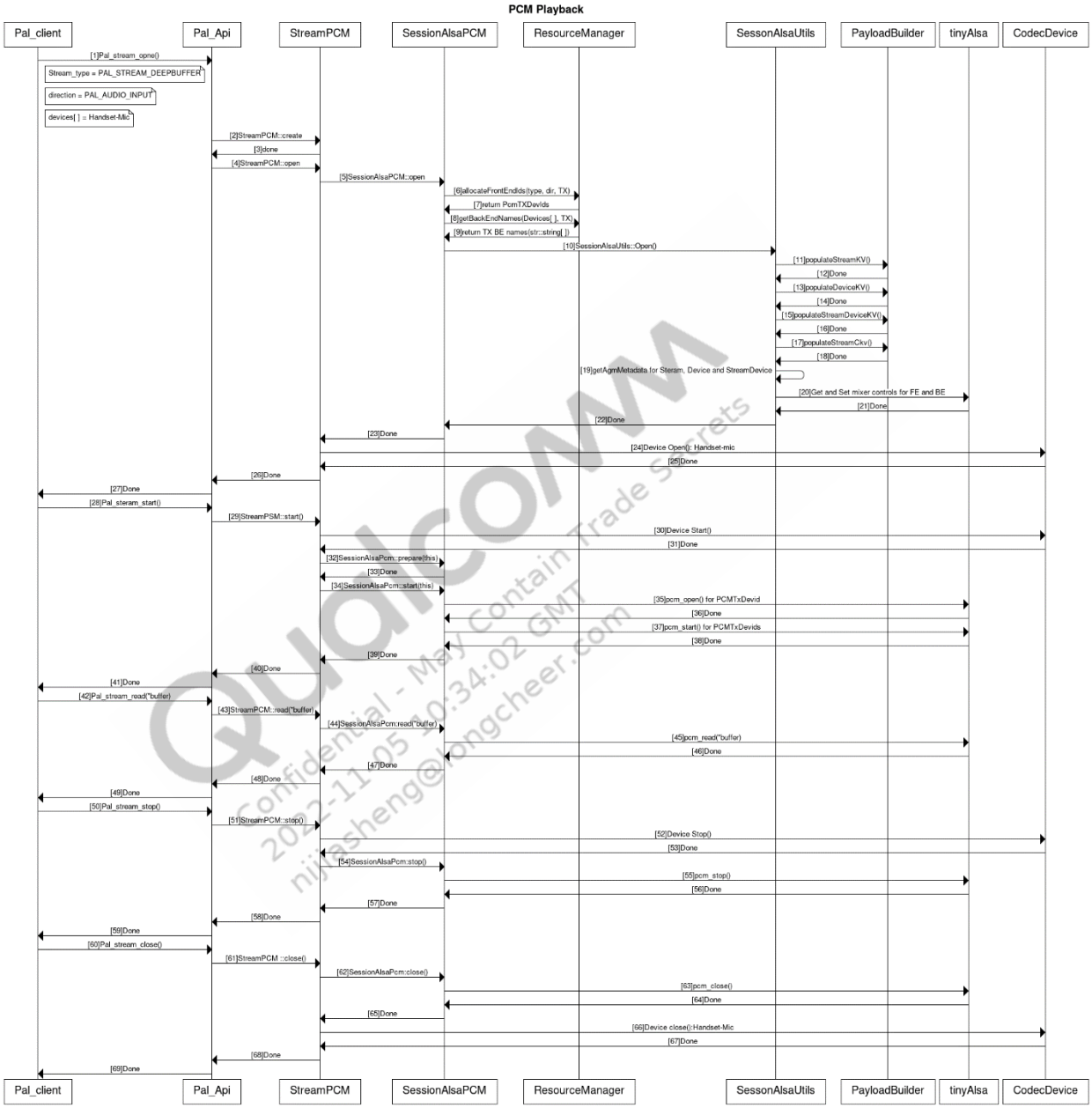
- Keeps track of all active sessions and devices to enable concurrencies
- Parses and loads configuration of platform related XML files
  - Resource_manager.xml – Device to back-end mapping, policy making attributes
  - Card-defs.xml – Available PCM and compress nodes and their capabilities
- Sound card parsing and mixer init

Stream PCM or stream compress is the entry point for playback; it controls the session and device. StreamAlsaPCM or StreamAlsaCompress sets up the PCM or compress sessions and controls them. The codec device sends enable or disable codec mixer controls and keeps track of the device reference count.

Payload Builder gets GKVs/CKVs for the given audio session and constructs custom payloads for modules in a given graph.

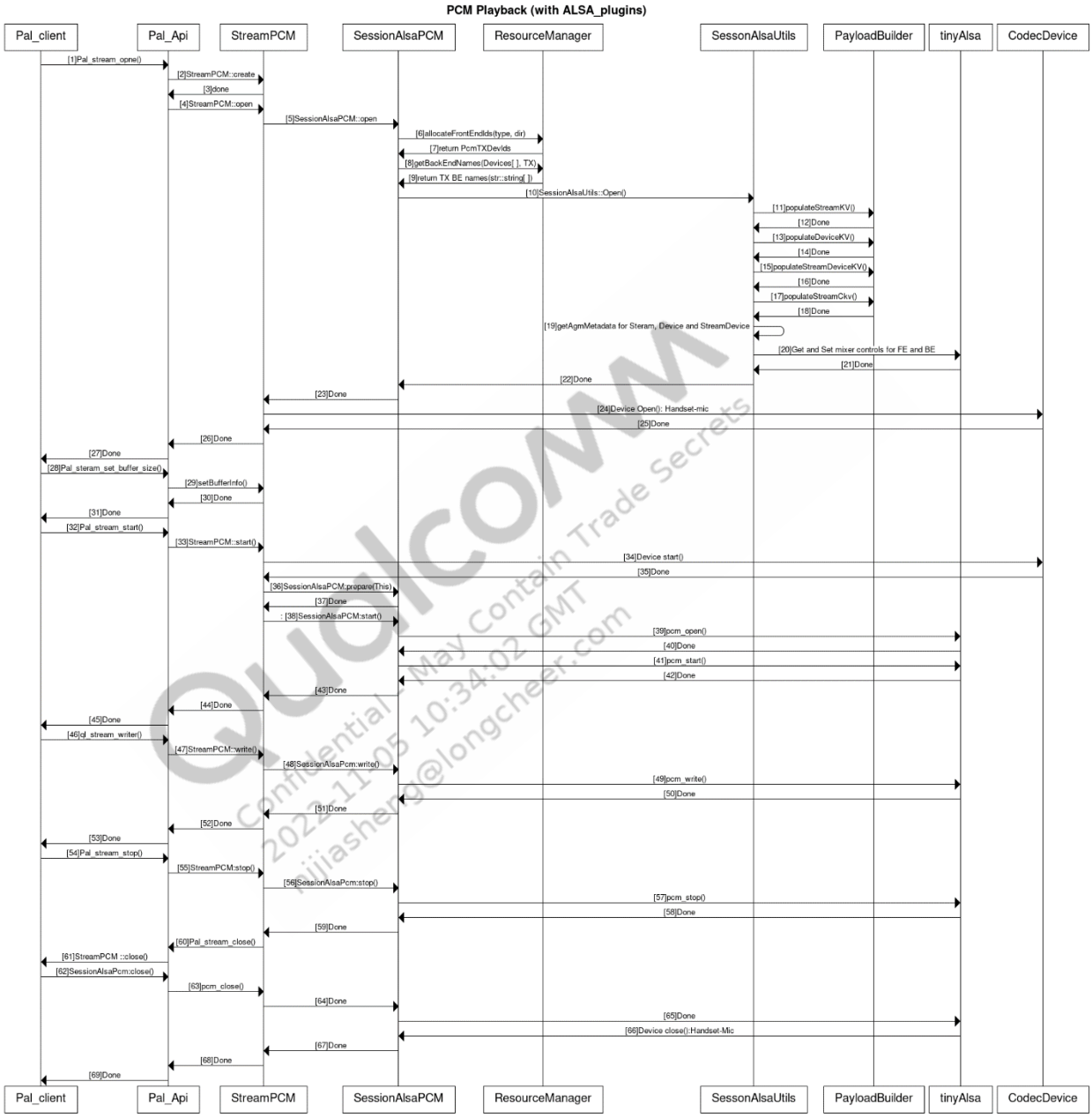
Qualcomm Confidential – May Contain Trade Secrets 2022-11-05 0:34:02 GMT sujitsher@lomx.qcom.com

The following is the call flow for the playback use case from PAL.



Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT nijasheng@longcheer.com

The following is the call flow for the record use case from PAL.

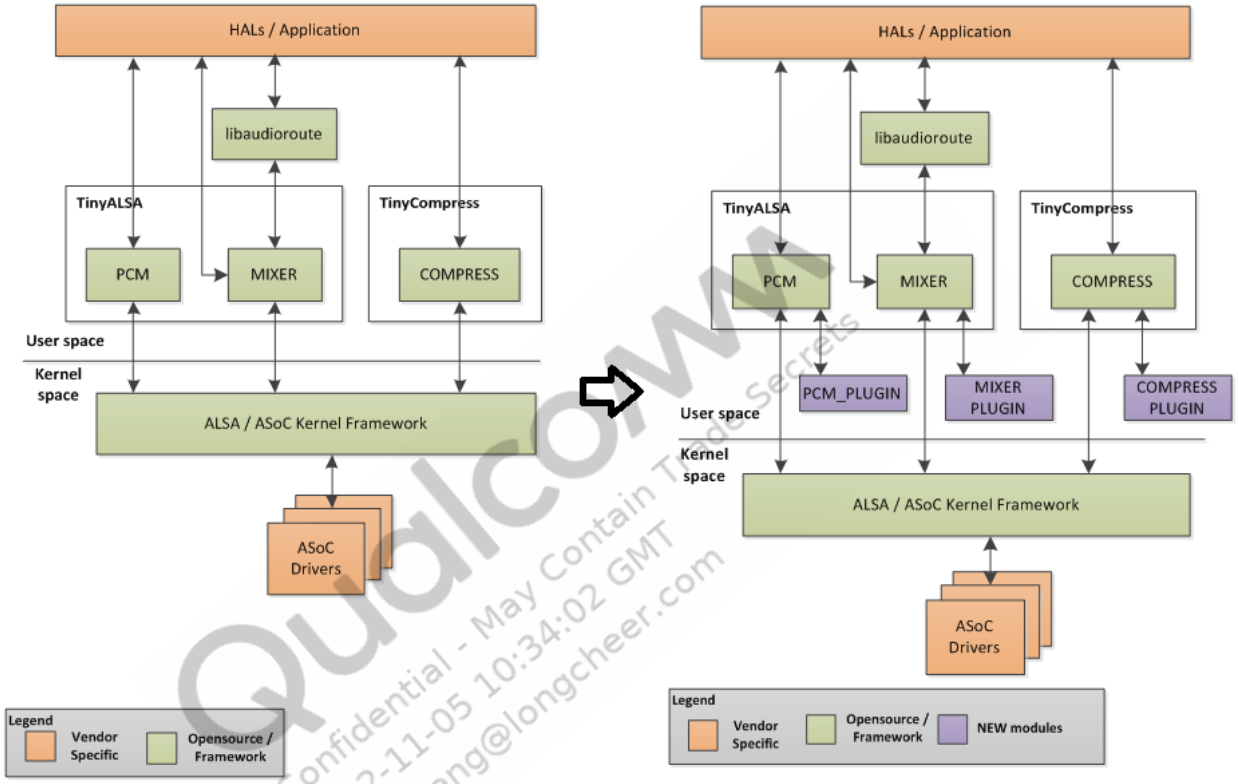


Qualcomm Confidential – May Contain Trade Secrets 2022-11-05 10:34:02 GMT rijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.3 ALSA plugins

TinyALSA is a library used to wrap the ALSA kernel interface into APIs that clients can invoke. The following figure depicts how the design changes from the Elite framework to AudioReach.



Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT xiuzheng@longcheer.com

Instead of Tiny[Alsa|Compress] always calling the kernel interface, it could optionally call the plugin interface. The plugin has an interface like that of the kernel interface.

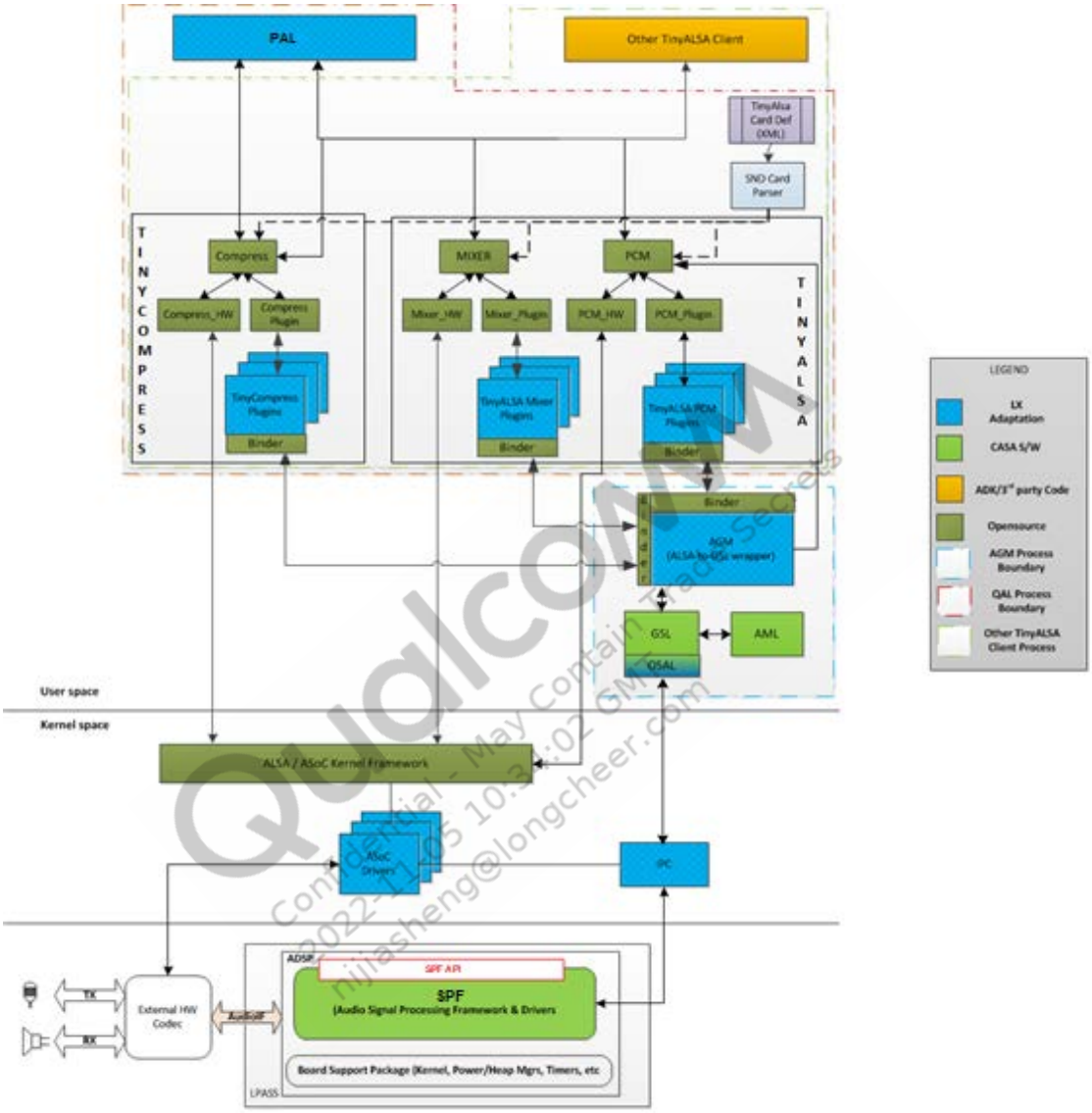
The ALSA framework in the kernel exposes card and device nodes for PCM/compress/mixer (physical card). Physical nodes exposed under /dev/snd/*. Plugins are realized as virtual nodes enumerated on a virtual sound card. Virtual nodes are mapped to on-device .so files (dynamically loadable shared objects). Plugins add virtual nodes into the existing physical sound card to create a completely virtual sound card with virtual nodes.

TinyALSA plugins perform the following tasks:

- Enumerates virtual mixer controls and virtual PCM/compressed devices
- Translates mixer controls to AGM API calls
- Translates PCM/compress operations to AGM API calls

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The following diagram explains the interaction of the involved modules.



PCM, mixer, compress:

- Upon open, gets the card definition from cards-defs-parser
- Opens an instance of either a hardware (physical) or plugin (virtual) device node
- Routes the operations to either hardware ops or plugin ops based on type

PCM_HW, MIXER_HW, COMPR_HW:

- Implements all operations to interface with the established ALSA kernel ops
- Reuses existing code from PCM/mixer/compress

PCM_PLUGIN, MIXER_PLUGIN, COMPR_PLUGIN:

- Loads dynamic (.so) libraries for each virtual device node
- Routes all PCM/mixer calls from the application to plugin-specific implementation

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Plugins:

- Compiled as .so libraries, implements function ops mandated by framework
- Could perform custom implementation of processing/routing of audio data

2.4 Snd-card-def parser

Snd-card-def parser is a library to parse the card definition XML and provide APIs to pcm.c and mixer.c to interface with card-defs.xml.

Cards_defs.xml contains the definitions of virtual nodes (PCMs/mixers/compress) devices. It can define virtual cards as well as add virtual nodes to an existing physical card.

The following are the properties that need to be defined in cards_def.xml:

- Card
  - id – card ID
  - name – Sound card name to be matched
- PCM device/compress device
  - id – PCM/compress device ID
  - name – Name of the PCM/compress device
  - so-name – Plugin file name for PCM/compress device
  - Properties
    - capture – If PCM/compress device has capture capability
    - playback – If PCM /compress device has playback capability
    - hostless – If PCM/compress device has hostless capability
- Mixer
  - id – Mixer ID
  - name – Name of the mixer
  - so-name – Plugin file name for mixer

2.5 Audio graph manager (AGM)

AGM is an audio service that runs in the Linux user space. AGM provides interfaces to allow TinyALSA-based mixer controls and PCM/compress plugins to interact and enable various audio use cases.

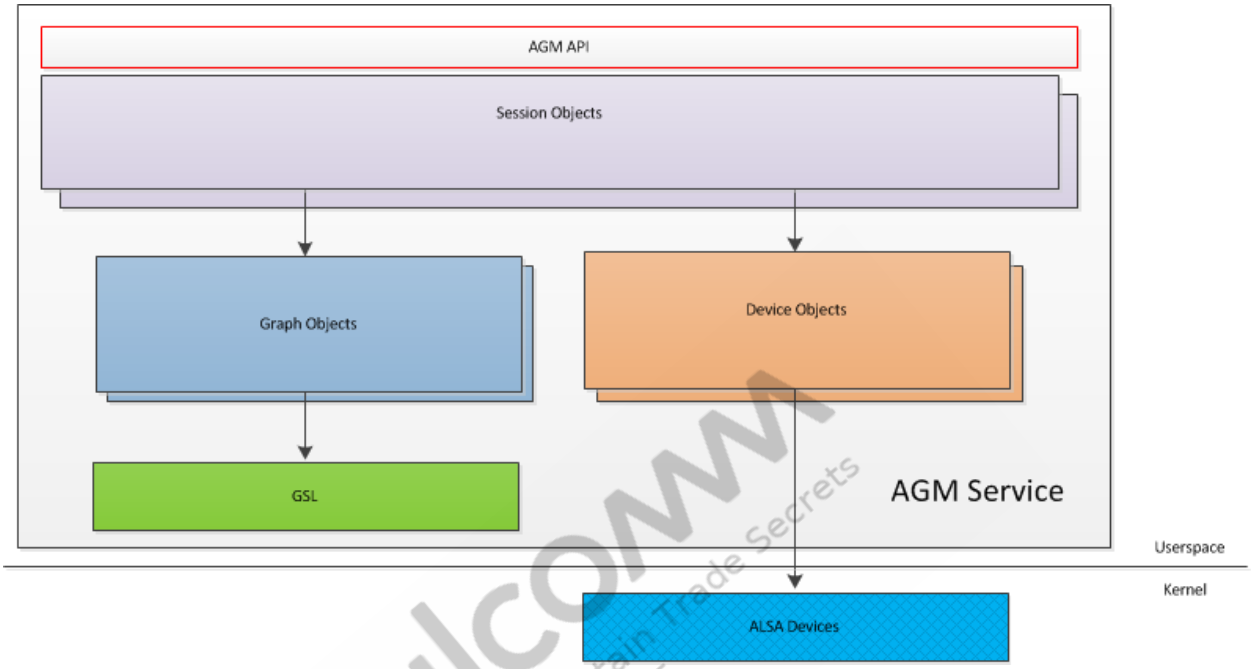
AGM performs the following operations:

- Sets up the graph service library (GSL) session with GKVs and CKVs
- Configures GSL buffers for sending or receiving audio samples
- Configures the encoder/decoder and hardware endpoints (I2S, TDM, SLIMbus, etc.)

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:22 GMT nijiaheng@longchee.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The following diagram shows the AGM block at a high level.



AGM provides APIs for mixer plugins and PCM/compress APIs to set up audio use cases. The AGM service runs as an independent service and maintains multiple ALSA clients to set up use cases. AGM also handles management of front-end to back-end connections.

The following subsections provide detailed explanations of AGM objects.

2.5.1 Session object

A session object represents an audio playback or capture session. A session object is created whenever a mixer control specific to a session is invoked or session-specific APIs are invoked. Once created, the session object is added to the session pool maintained by the AGM service.

Session objects have the following roles and responsibilities:

- Provide APIs to TinyALSA plugins to realize stream configuration
- Create graph and device objects and provide the required information to these objects
- Manage state transition of graph and device objects

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.5.2 Graph object

A graph object represents a graph (that is, an interconnection of modules running in the SPF). Graph objects interact with GSL to open, manage, and close graphs in the SPF.

Graph objects have the following roles and responsibilities:

- Provide APIs for creating, adding, and removing graphs
- Interact with GSL to create and manage graphs in the SPF
- Configure the following modules to enable ALSA APIs:
  - Stream – Sharedmem, decoder/encoder (PCM and compressed), PCM converter
  - Device – Hardware endpoint

2.5.3 Device object

A device object represents an ALSA device as enumerated by the ALSA sound card. Each ALSA device is a direct back-end DAI link that sets up kernel resources such as GPIOs, clocks, and codec driver

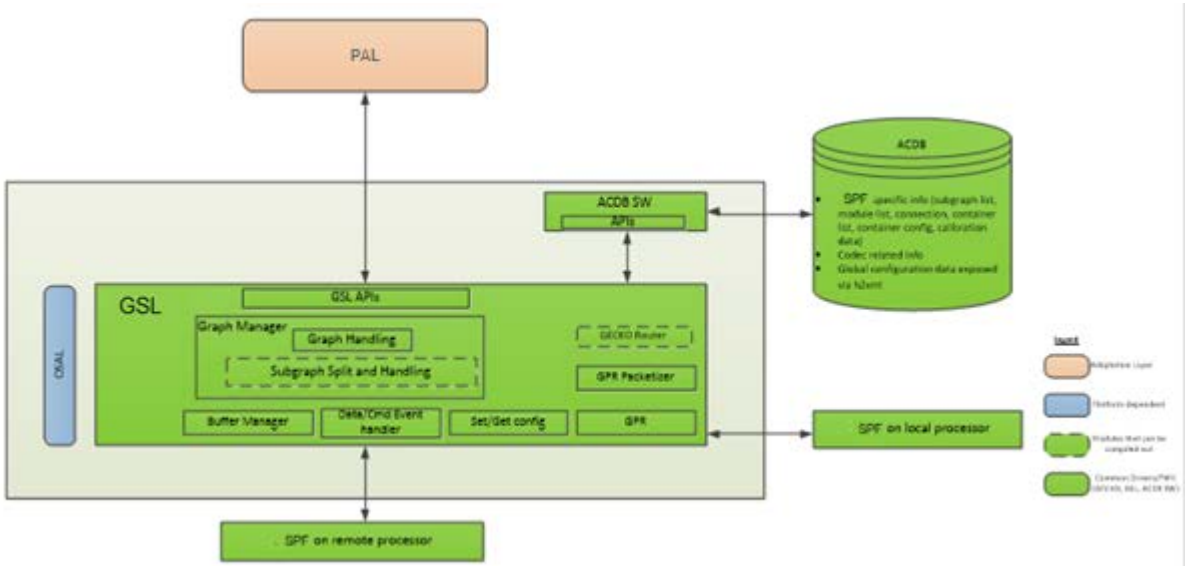
Device objects have the following roles and responsibilities:

- Enumerate available audio interfaces by reading the ALSA soundcard
- Provide device APIs for transitioning device states
- Manages ALSA back-end states with ref count. The ALSA back-end is configured with a client-specified media configuration.

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 14:34:02 GMT hsheng@longshear.com

2.6 GSL

GSL is a common software driver for the SPF framework. GSL is comprised of multiple software blocks including the API layer, graph and subgraph management, buffer management, event processing, set/get configuration, and OS abstraction layer. The following figure explains these blocks.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

GSL performs the following operations:

- Queries graph definitions and calibration from the ACDB manager library (AML) using the GKVs and CKVs
- Sends graph definitions and calibration to the SPF
- Sets up buffers to send and receive audio buffers

The following subsections provide detailed explanations of software GSL blocks/layers.

2.6.1 GSL API layer

The GSL API layer defines and exposes APIs to receive information on streams, devices, metadata, and buffers. It supports providing time stamps, buffer flags, sending events back to an application, postprocessing blocks, stream, postprocessing information, etc.

2.6.2 Graph handling

This block invokes APIs provided by the AML. It passes on the keys to get graph/subgraph definitions for the use case, which then routes a stream to a specific device.

2.6.3 Subgraph and concurrency handling

This block tracks the use case subgraphs that are currently active on the SPF and when new use cases need to be established. It only sends the delta subgraph information required to enable a given use case.

2.6.4 Set/get config

This layer performs functions to send module-specific calibration parameters and configurations (for example, media format, etc.) to modules in the SPF.

2.6.5 Buffer management

This layer sends and receives buffers to and from the SPF.

2.6.6 Event processing

This layer performs functions related to receiving events (for example subsystem restart (SSR), processing domain restart (PDR), end-of-stream (EOS), etc.) from the SPF. Some events are consumed within GSL itself and some of the events are passed to upper layers using callbacks.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.6.7 SPF router

This layer is responsible for routing the graph construction to the right instance of the SPF running on different processor (for example, ARM or DSP). In scenarios where DSP MIPS are limited, certain use cases can be supported with SPF running on ARM.

2.7 AML

AML exposes APIs to enable clients to get binary blobs for graph components (for example, subgraphs, components, modules, edges, etc.). AML APIs can be called either from GSL and PAL.

For example, APIs can be called on applications interacting directly with GSL for the following cases:

- To get custom parameters for H2XML-based configuration. This requires querying certain module related calibration parameters to form a SPF-understandable calibration packet. This is useful for UI custom module configurations and other frameworks like the generic effects frameworks (GEF) on Linux Android.
- Device management and control within PAL can directly call AML APIs to retrieve codec calibration data

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT nijiasheng@longcheer.com

3 AudioReach playback use cases

AudioReach has the same set of playback use cases available as in the prior framework. The following table lists the different playback use cases with mapping from HAL flags to PAL stream types.

Flags	AudioReach use case mapping
AUDIO_OUTPUT_FLAG_COMPRESS_OFFLOAD	PAL_STREAM_COMPRESSED
AUDIO_OUTPUT_FLAG_DIRECT	PAL_STREAM_PCM_OFFLOAD
AUDIO_OUTPUT_FLAG_DEEP_BUFFER	PAL_STREAM_DEEP_BUFFER
AUDIO_OUTPUT_FLAG_FAST AUDIO_OUTPUT_FLAG_PRIMARY	PAL_STREAM_LOW_LATENCY
AUDIO_OUTPUT_FLAG_RAW AUDIO_OUTPUT_FLAG_MMAP_NOIRQ	PAL_STREAM_ULTRA_LOW_LATENCY

NOTE: There can be many flags mapped to a particular use case. This table lists just a few.

Every use case is defined as a GKV consisting of the following items:

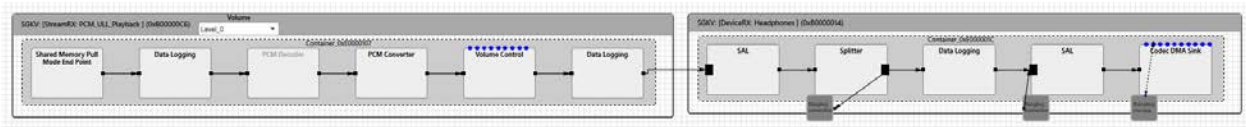
- Stream – Consists of shared memory endpoint, decoder, encoder, PCM converter, packetizer, and depacketizer. This is formed based on the pal_stream_type and pal_device from the HAL.
- Stream-PP – Consists of stream-specific postprocessing modules
- PSPD – Consists of any per-stream per-device modules such as channel mixer, MFC, etc.
- Device-PP – Consists of device-specific postprocessing modules. This is formed based on the key value pairs in resource_manager.xml. This is decided based on the snd device name.
- Device – Hardware endpoint. This is formed based on the pal_stream_type and pal_device from the HAL.

Qualcomm Confidential – May Contain Trade Secrets 2022-11-09 10:04:02 GMT yijiazheng@gmail.com

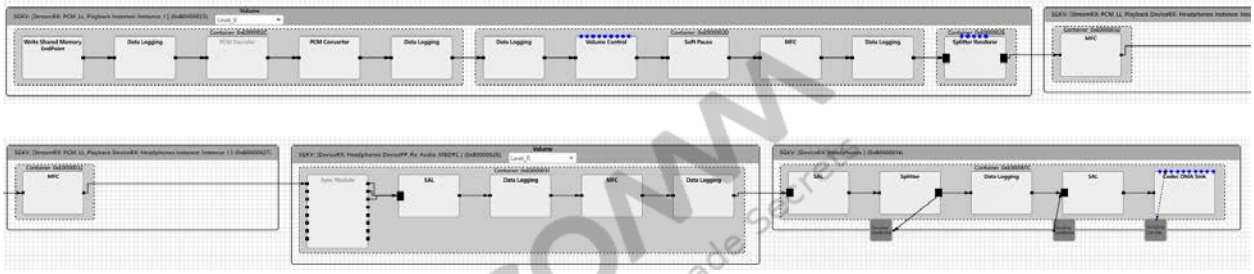
MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The following are the GKVs for various playback use cases.

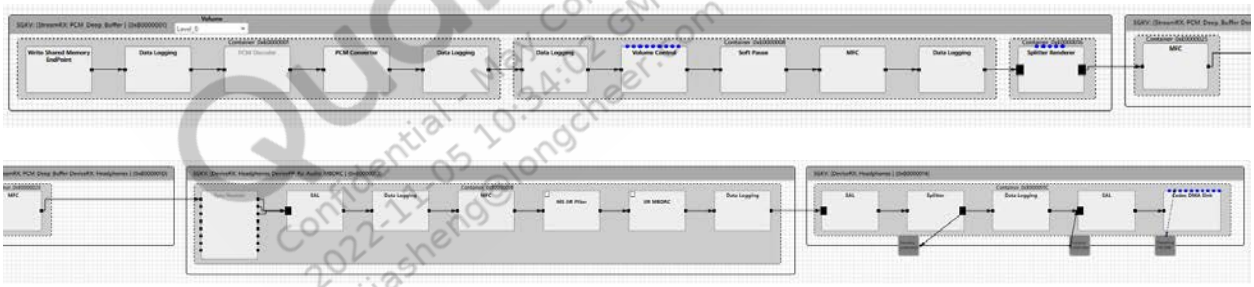
Ultra-low latency playback on headset



Low latency playback on headset

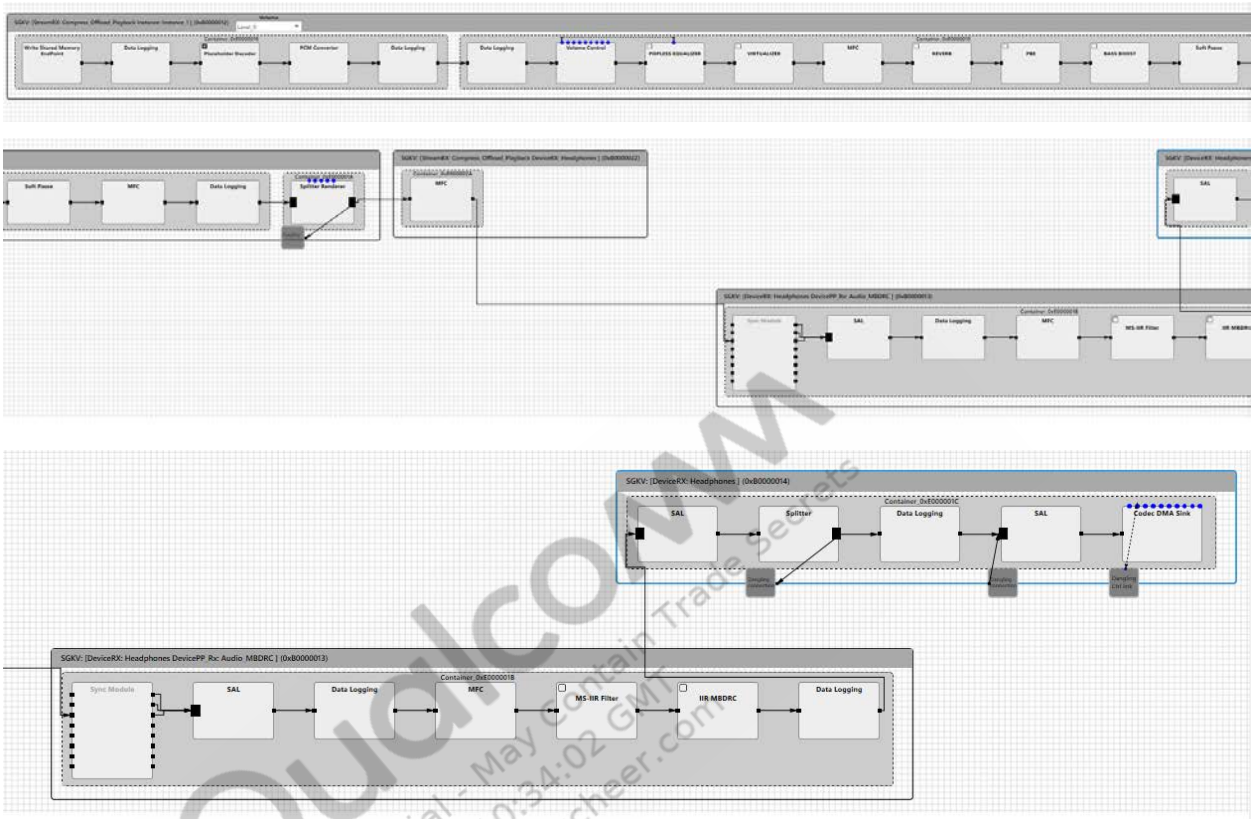


Deep buffer playback on headset



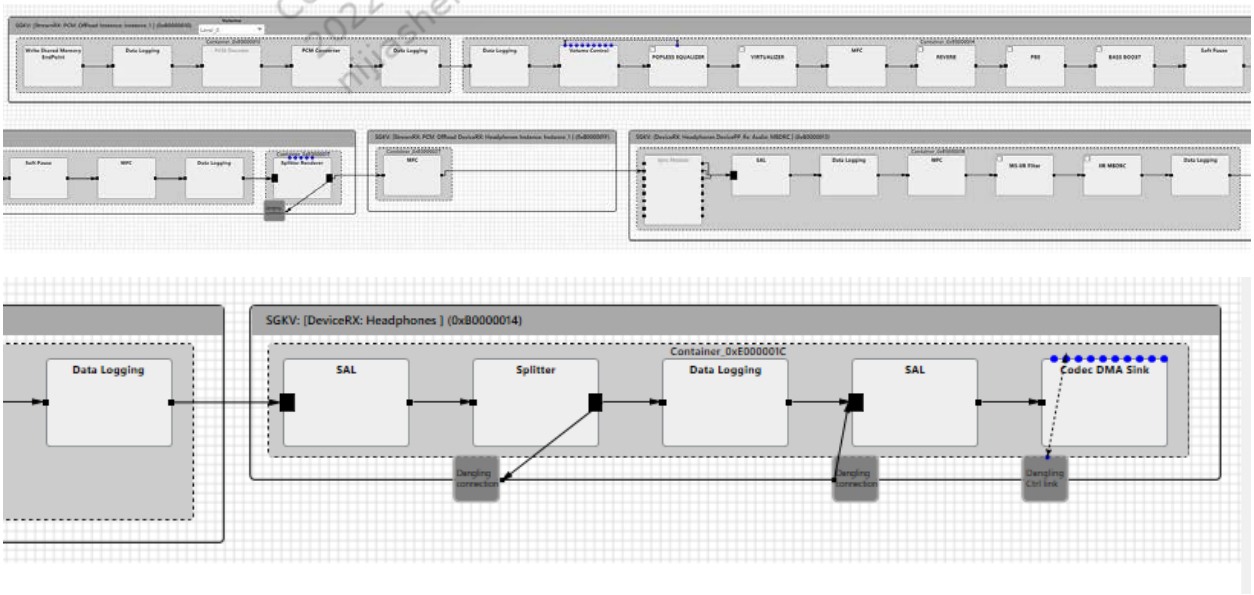
Qualcomm Confidential – May Contain Trade Secrets 2022-11-05 10:34:02 GMT nijiashengelongchaer.com

Compress offload playback on headset



Qualcomm Confidential - May Contain Trade Secrets 2022-12-05 10:34:02 GMT nikicheng@longcheer.com

PCM offload playback on headset

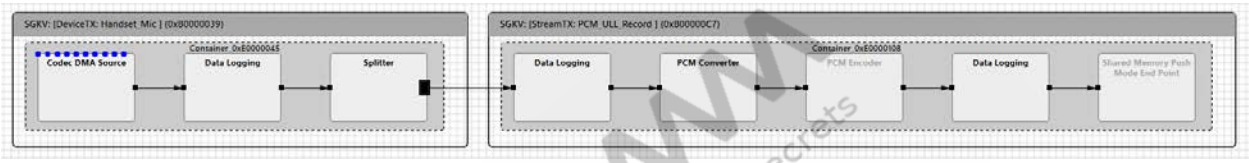


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

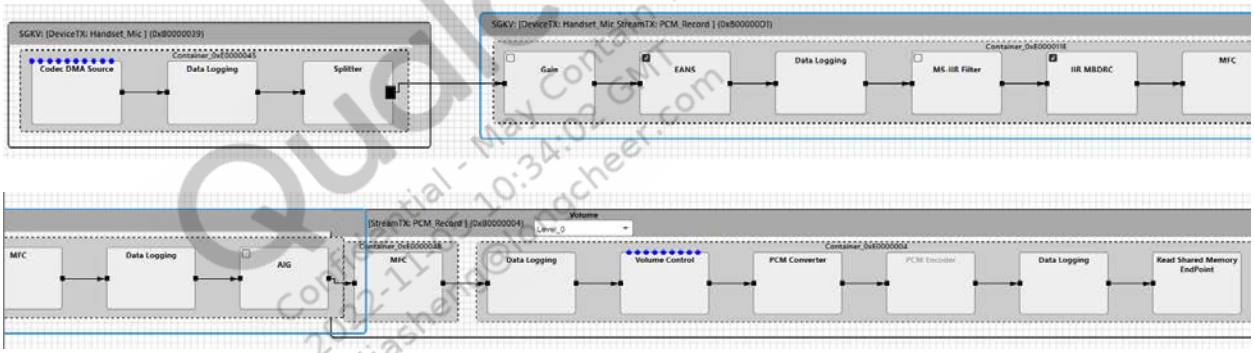
4 AudioReach record use cases

Similar to playback use cases, record use cases are represented by GKVs. The following are the GKVs for various record use cases.

ULL record



Compress record



Raw record

Qualcomm Confidential - May Contain Trade Secrets 2022-11-09 10:34:02 GMT nijiashen@qracheer.com

5 AudioReach debugging

5.1 Definitions

Most definitions and mappings of the device/stream/key/value are defined in the following files that can be referred to for log analysis:

- ResourceManager.xml
- PALDefs.h
- CardDefs.xml
- Kvh2xml.h

5.2 Dynamic logging

Verbose and debug logging can be enabled in modules by modifying the audio_dynamic_log.xml file. For example, to enable dynamic logging in device.c and graph.c, the following change is required. After changing the audio_dynamic_log.xml file, it should be pushed to the /vendor/etc/ folder.

```
--          <file name="device.c" log_enable="false"/>
++          <file name="device.c" log_enable="true"/>
…
--          <file name="graph.c" log_enable="false"/>
++          <file name="graph.c" log_enable="true"/>
```

Usage:

```
/* edit the xml file to either enable logs in all files under a module or
enable/disable logging in a file under a module
*/
$adb pull /vendor/etc/audio_dynamic_log.xml
/* After modification, push the xml file back to same location. */
$adb push <from_local_folder> /vendor/etc/
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 PST rui.zhengcheng@longcheer.com

5.3 AGM command line

AGM PCM playback can be tested with the following commands:

```
/*Commands to enable Codec*/
tinymix "WSA RX1 MUX" "AIF1_PB"
tinymix "WSA_RX1 INP0" "RX1"
tinymix "WSA_COMP2 Switch" "1"
tinymix "SpkrRight COMP Switch" "1"
tinymix "SpkrRight BOOST Switch" "1"
tinymix "SpkrRight VISENSE Switch" "1"
tinymix "SpkrRight SWR DAC_Port Switch" "1"
/*agm command to start pcm playback on codec*/
agmplay /data/test.wav -D 0 -d 100 -i CODEC_DMA-LPAIF_WSA-RX-0
```

AGM compress playback can be tested using the following commands:

```
/*Commands to enable Codec*/
tinymix "WSA RX1 MUX" "AIF1_PB"
tinymix "WSA_RX1 INP0" "RX1"
tinymix "WSA_COMP2 Switch" "1"
tinymix "SpkrRight COMP Switch" "1"
tinymix "SpkrRight BOOST Switch" "1"
tinymix "SpkrRight VISENSE Switch" "1"
tinymix "SpkrRight SWR DAC_Port Switch" "1"
/*agm command to start compress playback on codec*/
agmcompressplay /data/test.mp3 -c 100 -d 105 -t 1 -i CODEC_DMA-LPAIF_WSA-
RX-0
```

Capture using AGM commands can be tested using the following commands:

```
/*Commands to enable DMIC0 path on codec*/
tinymix 'TX DEC0 MUX' 'MSM_DMIC'
tinymix 'TX DMIC MUX0' 'DMIC0'
tinymix 'TX_CDC_DMA_TX_3 Channels' 'One'
tinymix 'TX_CDC_DMA_TX_3 SampleRate' 'KHZ_48'
tinymix 'TX_CDC_DMA_TX_3 Format' 'S16_LE'
tinymix 'TX_AIF1_CAP Mixer DEC0' 1
tinymix 'TX_DEC0 Volume' 84
/*agm command to start record*/
adb shell agmcap /data/rec.wav -D 100 -d 101 -c 2 -r 48000 -b 16 -i
CODEC_DMA-LPAIF_RXTX-TX-3
```

Qualcomm Confidential May Contain Trade Secrets 2022-11-15 10:24:02 GMT nijiashengsongcheer.com

5.4 GSL commands tool

To check GSL commands sent to the ADSP, the GPR sniffer tool is used. Refer to *GPR Packet Sniff Tool* (80-VN500-12) for more information.

5.5 Log analysis

The following are logs snippets of playback and record use cases that can be referred to get a better understanding of the use case.

Playback

The following log snippet explains compress offload playback over the headphone path.

```
//pal_stream_open is called first by HAL to start the playback
12-01 19:13:11.271   872  7366 I PAL: API: pal_stream_open: 130: Enter,
stream type:3
//Device ID 5 for Wired Headphone
12-01 19:13:11.271   872  7366 I PAL: Device: setDeviceAttributes: 242:
DeviceAttributes for Device Id 5 updated
//front End 105 for compress playback defined in card-defs.xml
12-01 19:13:11.271   872  7366 I PAL: ResourceManager: allocateFrontEndIds:
3263: allocateFrontEndIds: front end 105
//Mixer Controls are dynamically created
12-01 19:13:11.272   900  6333 D AGM: metadata: metadata_print: 77
************************Metadata************************
12-01 19:13:11.272   900  6333 D AGM: metadata: metadata_print: 78 GKV
size:2
//Key: StreamRX, Value: COMPRESSED_OFFLOAD_PLAYBACK from kvh2xml.h
12-01 19:13:11.272   900  6333 D AGM: metadata: metadata_print: 81
key:0xa1000000, value:0xa100000a
//Key: Instance, Value: 1
12-01 19:13:11.272   900  6333 D AGM: metadata: metadata_print: 81
key:0xab000000, value:0x1
12-01 19:13:11.272   900  6333 D AGM: metadata: metadata_print: 85
12-01 19:13:11.272   900  6333 D AGM: metadata: CKV count:1
//Key: Volume, Value: 15
12-01 19:13:11.272   900  6333 D AGM: metadata: metadata_print: 88
key:0xa4000000, value:0x15
12-01 19:13:11.272   900  6333 D AGM: metadata: metadata_print: 90
//Property ID: Sub Graph property Id: 0x08000010, Value: Stream from
gsl_subgraph_platform_driver_props.xml
12-01 19:13:11.272   900  6333 D AGM: metadata: metadata_print: 92 Property
ID:134217744
12-01 19:13:11.272   900  6333 D AGM: metadata: metadata_print: 93 property
count:1
12-01 19:13:11.272   900  6333 D AGM: metadata: metadata_print: 96 Property
Values:
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
12-01 19:13:11.272   900  6333 D AGM: metadata: metadata_print: 98 0x1
12-01 19:13:11.273   900  6333 D AGM: metadata: metadata_print: 100
12-01 19:13:11.273   900  6333 D AGM: metadata: metadata_print: 101
****************Metadata Done***************************
12-01 19:13:11.273   900  6333 D AGM: metadata: metadata_print: 77
*************************Metadata*************************
12-01 19:13:11.273   900  6333 D AGM: metadata: metadata_print: 78 GKV
size:1
//Key: DeviceRX, Value: Headphones
12-01 19:13:11.273   900  6333 D AGM: metadata: metadata_print: 81
key:0xa2000000, value:0xa2000002
12-01 19:13:11.273   900  6333 D AGM: metadata: metadata_print: 85
12-01 19:13:11.273   900  6333 D AGM: metadata: CKV count:0
12-01 19:13:11.273   900  6333 D AGM: metadata: metadata_print: 90
//Property Id: Subgraph Property ID: 0x80000010, Value: Device
12-01 19:13:11.273   900  6333 D AGM: metadata: metadata_print: 92 Property
ID:134217744
12-01 19:13:11.273   900  6333 D AGM: metadata: metadata_print: 93 property
count:2
12-01 19:13:11.273   900  6333 D AGM: metadata: metadata_print: 96 Property
Values:
12-01 19:13:11.273   900  6333 D AGM: metadata: metadata_print: 98 0x2
12-01 19:13:11.273   900  6333 D AGM: metadata: metadata_print: 98 0x5
12-01 19:13:11.273   900  6333 D AGM: metadata: metadata_print: 100
12-01 19:13:11.273   900  6333 D AGM: metadata: metadata_print: 101
****************Metadata Done*************************** 12-01 19:13:11.274
900  6333 D AGM: metadata: metadata_print: 77
*************************Metadata*************************
12-01 19:13:11.274   900  6333 D AGM: metadata: metadata_print: 78 GKV
size:2
//Key: DevicePP_Rx, Value: DevicePP_Rx_AUDIO_MBDRC
12-01 19:13:11.274   900  6333 D AGM: metadata: metadata_print: 81
key:0xac000000, value:0xac000002
12-01 19:13:11.274   900  6333 D AGM: metadata: metadata_print: 81
key:0xa2000000, value:0xa2000002
12-01 19:13:11.274   900  6333 D AGM: metadata: metadata_print: 85
12-01 19:13:11.274   900  6333 D AGM: metadata: CKV count:1
//Key: GAIN, Value: 0
12-01 19:13:11.274   900  6333 D AGM: metadata: metadata_print: 88
key:0xd2000000, value:0x0
12-01 19:13:11.274   900  6333 D AGM: metadata: metadata_print: 90
//Property Id: Subgraph Property ID: 0x80000010, Value: StreamDevice
12-01 19:13:11.274   900  6333 D AGM: metadata: metadata_print: 92 Property
ID:134217744
12-01 19:13:11.274   900  6333 D AGM: metadata: metadata_print: 93 property
count:1
12-01 19:13:11.275   900  6333 D AGM: metadata: metadata_print: 96 Property
Values:
12-01 19:13:11.275   900  6333 D AGM: metadata: metadata_print: 98 0x3
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
12-01 19:13:11.275   900  6333 D AGM: metadata: metadata_print: 100
12-01 19:13:11.275   900  6333 D AGM: metadata: metadata_print: 101
****************Metadata Done***************************
//Format: AUDIO_FORMAT_MP3
12-01 19:13:11.275   872  7366 D AHAL: AudioStream:
get_compressed_buffer_size: 1840: config_ 1000000
//Size of Buffer: 32*1024, Number of buffers: 4
12-01 19:13:11.275   872  7366 D AHAL: AudioStream: Open: 2063:
fragment_size_ 32768 fragments_ 4
12-01 19:13:11.276   872  7366 E PAL: StreamCompress: start: 320: device 5
name PAL_DEVICE_OUT_WIRED_HEADPHONE, going to start
//Backend Configuration set by HAL
12-01 19:13:11.286   872  7366 I PAL: SessionAlsaUtils:
setDeviceMediaConfig: 685: CODEC_DMA-LPAIF_RXTX-RX-0 rate ch fmt data_fmt
48000 2 2 1
//AGM session open called
12-01 19:13:11.289   872  7366 E agm_client_wrapper: agm_session_open
called with handle = e7bfab60 , *handle = 0
//graph_open called with specific miid and tag
12-01 19:13:11.291   900  1794 D AGM: graph: graph_open: 424 entry
//MIID: module instance id, MID: Module Id queried from ACDB, tag: module
tag defined in the platform header file exported to ACDB
//mid: MID_WR_SHARED_MEM_EP, Tag: input_media_format
12-01 19:13:11.293   900  1794 D AGM: graph: graph_open: 491 miid 4077 mid
7001000 tag c0000002
//mid: Codec_DMA_Sink, Tag: hw_ep_rx
12-01 19:13:11.293   900  1794 D AGM: graph: graph_open: 536 miid 43b2 mid
7001023 tag c0000004
//mid: MID_SOFT_PAUSE, Tag: Pause
12-01 19:13:11.293   900  1794 D AGM: graph: graph_open: 491 miid 407e mid
7001019 tag c0000006
//mid: MID_PCM_CONVERTER, Tag: stream_pcm_convertor
12-01 19:13:11.293   900  1794 D AGM: graph: graph_open: 491 miid 407a mid
7001003 tag c0000010
//mid:PlaceHolderDecoder, Tag: stream_placeholder_decoder
12-01 19:13:11.293   900  1794 D AGM: graph: graph_open: 491 miid 4082 mid
7001009 tag c0000012
//mid: SpliterRendrer, Tag: Stream_Spr
12-01 19:13:11.293   900  1794 D AGM: graph: graph_open: 491 miid 4164 mid
7001032 tag c0000013
//graph_open called with handle 0x47534c used for further operations
12-01 19:13:11.297   900  1794 D AGM: graph: graph_open: 569 graph_handle
0x47534c
12-01 19:13:11.304   900   900 D AGM: graph: graph_set_config: 888 entry
graph_handle 0x47534c
//graph_start called on 0x47534c
12-01 19:13:11.372   900   900 D AGM: graph: graph_start: 736 entry
graph_handle 0x47534c
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT nijasheng@ingchen.com

The following log snippet explains the stop sequence for an offload playback use case.

```
//Pause called from Audio HAL
12-01 19:13:18.728   872   916 D AHAL: AudioStream: Pause: 1490: Exit ret:
0
//graph_stop called on respective graph handle
12-01 19:13:21.966   900  7368 D AGM: graph: graph_stop: 764 entry
graph_handle 0x47534c
12-01 19:13:21.966   900  7368 D AGM: graph: graph_stop: 764 entry
graph_handle 0x47534c
//graph_close called
12-01 19:13:22.030   900  1795 D AGM: graph: graph_close: 605 entry handle
0x47534c
//AGM session is closed
12-01 19:13:22.038   900  1795 E AGM: session: session_close: 1166 exit
//ResourceManager frees the frontend and stream
12-01 19:13:22.040   872   914 I PAL: ResourceManager: freeFrontEndIds:
3374: stream type 3, freeing 105
12-01 19:13:22.041   872   914 E PAL: ResourceManager:
resetStreamInstanceID: 5138: Invalid Stream Instance ID
12-01 19:13:22.042   872   914 I PAL: ResourceManager: deregisterStream:
1633: stream type 3
//AudioStream enters standby
12-01 19:13:22.042   872   914 D AHAL: AudioStream: Standby: 1587: Exit
ret: 0
12-01 19:13:22.042   872   914 D AHAL: AudioStream: astream_out_standby:
475: Exit ret: 0
//AHAL stream close called
12-01 19:13:22.045   872   916 D AHAL: AudioDevice:
adev_close_output_stream: 421: enter:stream_handle(0xea140b50)
12-01 19:13:22.045   872   916 D AHAL: AudioDevice:
adev_close_output_stream: 425: exit
12-01 19:13:22.045   872   916 D AHAL: AudioStream: ~StreamOutPrimary:
2457: close stream, handle(85), pal_stream_handle (0x0)
```

Record

The following log snippet explains the audio record use case for handset mic.

```
//Audio HAL sets the backend configuration for recording.
12-01 20:59:12.364   872   914 D AHAL: AudioDevice: adev_open_input_stream:
464: enter: sample_rate(8000) channel_mask(0x10) devices(0x80000004)
io_handle(78) source(1) format 1
12-01 20:59:12.364   872   914 D AHAL: AudioStream: StreamInPrimary: 3195:
enter: handle (4e) format(0x1) sample_rate(8000) channel_mask(0x10)
devices(1) flags(0)
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-07 10:31:02 GMT miasheng@longchuer.com

```
//Audio HAL sets the device ids. PalDeviceId: Wired headset
12-01 20:59:12.365   872   914 D AHAL: AudioDevice: GetPalDeviceIds: 1411:
Found haldeviceId: 80000004 and PAL Device ID 20
12-01 20:59:12.365   872   914 D AHAL: AudioDevice: CreateStreamIn: 308:
input stream 1 0xea4c791c
12-01 20:59:12.376   990  8814 I AudioFlinger: AudioFlinger's thread
0xb4000071be755040 tid=8814 ready to run
12-01 20:59:12.379  1296  2494 W AudioRecord: createRecord_l(50): Server
adjusted notificationFrames from 1024 to 160 for frameCount 4096
12-01 20:59:12.379   990   990 W AudioFlinger: acquireAudioSessionId()
unknown client 8761 for session 145
12-01 20:59:12.446   872   914 D AHAL: AudioDevice: CreateAudioPatch: 129:
enter: num sources 1, num_sinks 1
//Source Role: AUDIO_PORT_ROLE_SOURCE, source type: AUDIO_PORT_TYPE_DEVICE
12-01 20:59:12.446   872   914 D AHAL: AudioDevice: CreateAudioPatch: 144:
source role 1, source type 1
12-01 20:59:12.446   872   914 D AHAL: AudioDevice: CreateAudioPatch: 155:
Capture patch from device 80000004 to mix 78
12-01 20:59:12.446   872   914 D AHAL: AudioStream: RouteStream: 2735:
mAndroidInDevices -2147483644, mNoOfInDevices 1, new_devices -2147483644,
num new_devices: 1
12-01 20:59:12.456  8761  8809 I Recorder: requestAudioFocus true
12-01 20:59:12.460   872  8818 D AHAL: AudioStream: Open: 2823: Enter
InPrimary
//PAL starts the record usecase with pal_stream_open()
12-01 20:59:12.460   872  8818 I PAL: API: pal_stream_open: 130: Enter,
stream type:2
//Front End Is: 115 is used for capture
12-01 20:59:12.461   872  8818 I PAL: ResourceManager: allocateFrontEndIds:
3175: allocateFrontEndIds: front end 115
12-01 20:59:12.462   872  8818 D PLUGIN: mixer: amp_pcm_metadata_put: 931
amp_pcm_metadata_put: enter
12-01 20:59:12.462   900  1795 D AGM: metadata: metadata_print: 77
************************Metadata************************
12-01 20:59:12.462   900  1795 D AGM: metadata: metadata_print: 78 GKV
size:1
//Key: StreamTx, Value: PCM_Record
12-01 20:59:12.462   900  1795 D AGM: metadata: metadata_print: 81
key:0xb1000000, value:0xb1000001
12-01 20:59:12.462   900  1795 D AGM: metadata: metadata_print: 85
12-01 20:59:12.462   900  1795 D AGM: metadata: CKV count:1
//Key: Volume, Value: 15
12-01 20:59:12.462   900  1795 D AGM: metadata: metadata_print: 88
key:0xa4000000, value:0x15
12-01 20:59:12.462   900  1795 D AGM: metadata: metadata_print: 90
//Property Id: Subgraph Property ID: 0x80000010, Value: Stream
12-01 20:59:12.462   900  1795 D AGM: metadata: metadata_print: 92 Property
ID:134217744
```

Qualcomm Confidential – May Contain Trade Secrets 2022-11-05 10:54:02 GMT jijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
12-01 20:59:12.462   900  1795 D AGM: metadata: metadata_print: 93 property
count:1
12-01 20:59:12.463   900  1795 D AGM: metadata: metadata_print: 96 Property
Values:
12-01 20:59:12.463   900  1795 D AGM: metadata: metadata_print: 98 0x1
12-01 20:59:12.463   900  1795 D AGM: metadata: metadata_print: 100
12-01 20:59:12.463   900  1795 D AGM: metadata: metadata_print: 101
****************Metadata Done***************************
12-01 20:59:12.463   900  1795 D AGM: metadata: metadata_print: 77
*************************Metadata*************************
12-01 20:59:12.463   900  1795 D AGM: metadata: metadata_print: 78 GKV
size:1
//Key: DeviceTx, Value: Handset_Mic
12-01 20:59:12.463   900  1795 D AGM: metadata: metadata_print: 81
key:0xa3000000, value:0xa3000004
12-01 20:59:12.463   900  1795 D AGM: metadata: metadata_print: 85
12-01 20:59:12.463   900  1795 D AGM: metadata: CKV count:0
12-01 20:59:12.463   900  1795 D AGM: metadata: metadata_print: 90
//Property Id: Subgraph Property ID: 0x80000010, Value: Device
12-01 20:59:12.463   900  1795 D AGM: metadata: metadata_print: 92 Property
ID:134217744
12-01 20:59:12.464   900  1795 D AGM: metadata: metadata_print: 93 property
count:2
12-01 20:59:12.464   900  1795 D AGM: metadata: metadata_print: 96 Property
Values:
12-01 20:59:12.464   900  1795 D AGM: metadata: metadata_print: 98 0x2
12-01 20:59:12.464   900  1795 D AGM: metadata: metadata_print: 98 0x5
12-01 20:59:12.464   900  1795 D AGM: metadata: metadata_print: 100
12-01 20:59:12.464   900  1795 D AGM: metadata: metadata_print: 101
****************Metadata Done***************************
12-01 20:59:12.465   900  1795 D AGM: metadata: metadata_print: 77
*************************Metadata*************************
12-01 20:59:12.465   900  1795 D AGM: metadata: metadata_print: 78 GKV
size:2
//Key: DevicePP_Tx, Value: DEVICEPP_TX_AUDIO_FLUENCE_SMECNS
12-01 20:59:12.465   900  1795 D AGM: metadata: metadata_print: 81
key:0xad000000, value:0xad000002
//Key: DeviceTx, Value: HandsetMic
12-01 20:59:12.465   900  1795 D AGM: metadata: metadata_print: 81
key:0xa3000000, value:0xa3000004
12-01 20:59:12.465   900  1795 D AGM: metadata: metadata_print: 85
12-01 20:59:12.465   900  1795 D AGM: metadata: CKV count:0
12-01 20:59:12.466   900  1795 D AGM: metadata: metadata_print: 90
//Property Id: Subgraph Property ID: 0x80000010, Value: StreamDevice
12-01 20:59:12.466   900  1795 D AGM: metadata: metadata_print: 92 Property
ID:134217744
12-01 20:59:12.466   900  1795 D AGM: metadata: metadata_print: 93 property
count:1
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-15 10:34:02 GMT mysnapdragon@qualcomm.com

```
12-01 20:59:12.466   900  1795 D AGM: metadata: metadata_print: 96 Property
Values:
12-01 20:59:12.466   900  1795 D AGM: metadata: metadata_print: 98 0x3
12-01 20:59:12.466   900  1795 D AGM: metadata: metadata_print: 100
12-01 20:59:12.466   900  1795 D AGM: metadata: metadata_print: 101
****************Metadata Done***************************
//Backend config set by ALSA
12-01 20:59:12.476   872  8818 I PAL: SessionAlsaUtils:
setDeviceMediaConfig: 685: CODEC_DMA-LPAIF_RXTX-TX-3 rate ch fmt data_fmt
48000 1 2 1
//AGM Session open called
12-01 20:59:12.479   872  8818 E agm_client_wrapper: agm_session_open
called with handle = e7bfa7d8 , *handle = 0
//graph starts with graph_open
12-01 20:59:12.480   900  1795 D AGM: graph: graph_open: 424 entry
//MIID: module instance id, MID: Module Id queried from ACDB, tag: module
tag defined in the platform header file exported to ACDB
//mid: Codec_DMA_Source, Tag: device_hw_ep_tx
12-01 20:59:12.481   900  1795 D AGM: graph: graph_open: 536 miid 43af mid
7001024 tag c0000005
//mid: PCM Encoder, Tag: stream_pcm_encoder
12-01 20:59:12.481   900  1795 D AGM: graph: graph_open: 491 miid 400e mid
7001004 tag c000000f
//mid: PCM Converer, tag: stream_pcm_converter
12-01 20:59:12.481   900  1795 D AGM: graph: graph_open: 491 miid 400d mid
7001003 tag c0000010
//graph_config called on the same graph handle
12-01 20:59:12.501   900   900 D AGM: graph: graph_set_config: 888 entry
graph_handle 0x47534c
//graph_prepare called
12-01 20:59:12.502   900   900 D AGM: graph: graph_prepare: 649 entry
graph_handle 0x47534c
//graph_start called which will start the record usecase
12-01 20:59:12.521   900  1795 D AGM: graph: graph_start: 736 entry
graph_handle 0x47534c
```

Qualcomm Confidential – May Contain Trade Secrets 2022-11-05 10:24:02 GMT jijiashen@longcheer.com

The following log snippet explains the stop sequence for a record use case.

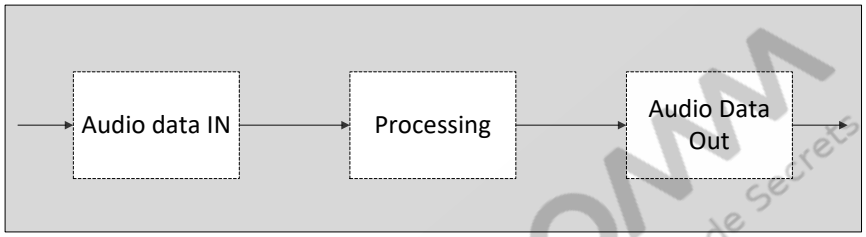
```
//MediaCodecSource calls encoding stop
12-01 20:59:27.256  1296  8812 I MediaCodecSource: encoder (audio) stopping
12-01 20:59:27.288   872   914 D AHAL: AudioStream: astream_in_standby:
1115: enter: stream (0xe95c4e70) usecase(20: audio-record)
//PAL Stream Stop call
12-01 20:59:27.288   872   914 E PAL: StreamPCM: stop: 506: In
PAL_AUDIO_INPUT case, device count – 1
//graph_stop called
12-01 20:59:27.292   900  1795 D AGM: graph: graph_stop: 764 entry
graph_handle 0x47534c
12-01 20:59:27.301   900  7368 D AGM: graph: graph_stop: 764 entry
graph_handle 0x47534c
//graph_close will release the graph
12-01 20:59:27.302   900  6333 D AGM: graph: graph_close: 605 entry handle
0x47534c
//AGM session will be closed with session_close
12-01 20:59:27.304   900  6333 E AGM: session: session_close: 1166 exit
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT nijiasheng@longcheer.com

6 SPF audio playback

6.1 Overview

Audio playback in the SPF can be generally represented by the following diagram consisting of three logical blocks.



6.1.1 Audio data IN

This logical block represents module(s) handling audio data input via packet exchange mode or shared circular/ring buffers(no packet exchange). Examples of this type of module are the write shared memory endpoint (WR SH Mem EP) module and the Pull mode EP module.

6.1.2 Processing

This logical block represents module(s) that handle audio processing operations. Example processing modules are decoders, postprocessing effects, media format converters, splitters, accumulators, etc.

6.1.3 Audio data OUT

This logical block represents module(s) handling audio data rendering onto devices such as sink devices. Example of this type of module are I2S, SLIMbus, codec DMA, TDM, etc.

6.2 Playback capabilities

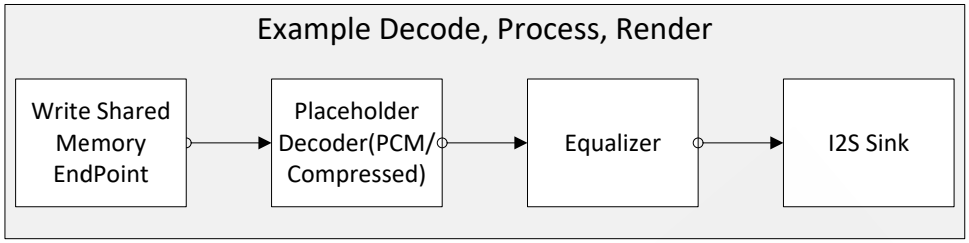
The capabilities of audio playback in SPF are categorized into three types:

- Based on the type of audio data supported in SPF
- Based on the data flow mechanism in SPF
- Based on session control capabilities facilitated in SPF

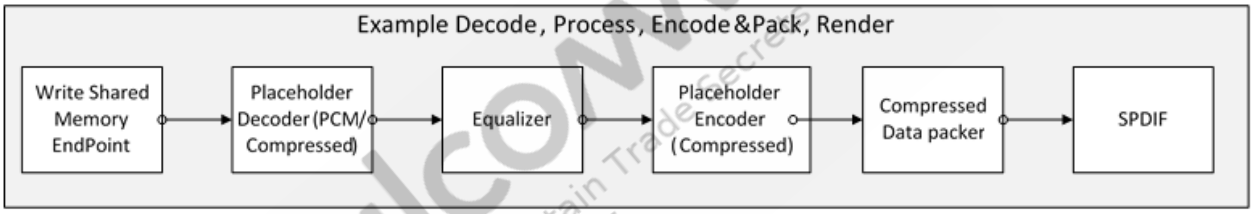
Qualcomm Confidential May Contain Trade Secrets 2022-11-05 10:51:03 GMT hanheng@longchuang.com

6.2.1 Data type based

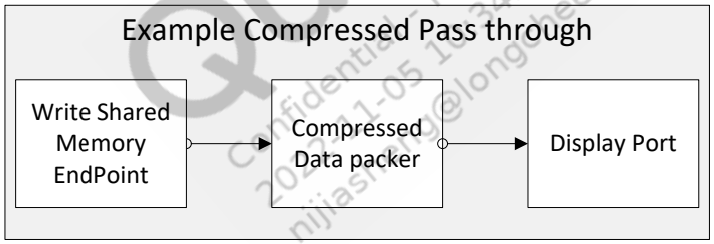
- Compressed/PCM data can be decoded and processed, and the resulting PCM data can be rendered onto sink devices.



- Compressed/PCM data can be decoded, processed, encoded, packed into compressed data transmission formats, and rendered onto sink devices.



- Compressed data can be packed into compressed data transmission formats and rendered onto sink devices.



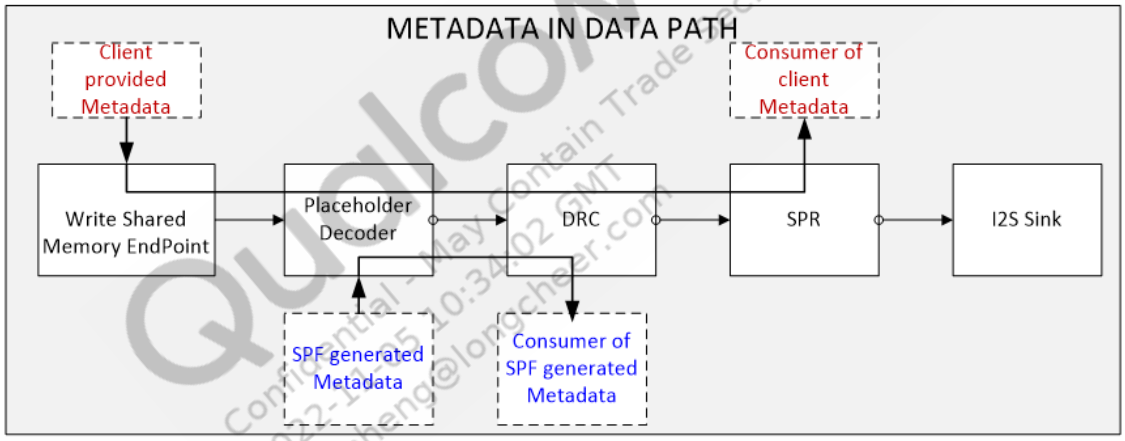
Qualcomm Confidential - May Contain Trade Secrets 2025-11-05 15:34:02 GMT nijiashang@longcheer.com

NOTE: These playback capabilities are also available as part of the previous Elite framework software architecture.

6.2.2 Data flow based

6.2.2.1 Metadata propagation

- Metadata from HLOS clients such as timestamped/non-timestamped buffers and buffer markers are propagated in the processing chain and finally consumed by a specific module in SPF. For example, the HLOS-provided timestamp data is received by the WR SH Mem EP module, propagated in the SPF chain, and is consumed by the SPR module that then uses the received timestamp data to perform rendering decisions.
- Metadata also could be generated by a module and consumed by another module in the processing chain within the SPF. The following diagram shows an example where the decoder module produces metadata such as DRC parameters based on an elementary bitstream which will then be consumed by the postprocessing DRC module along with the data to perform DRC gain adjustments.



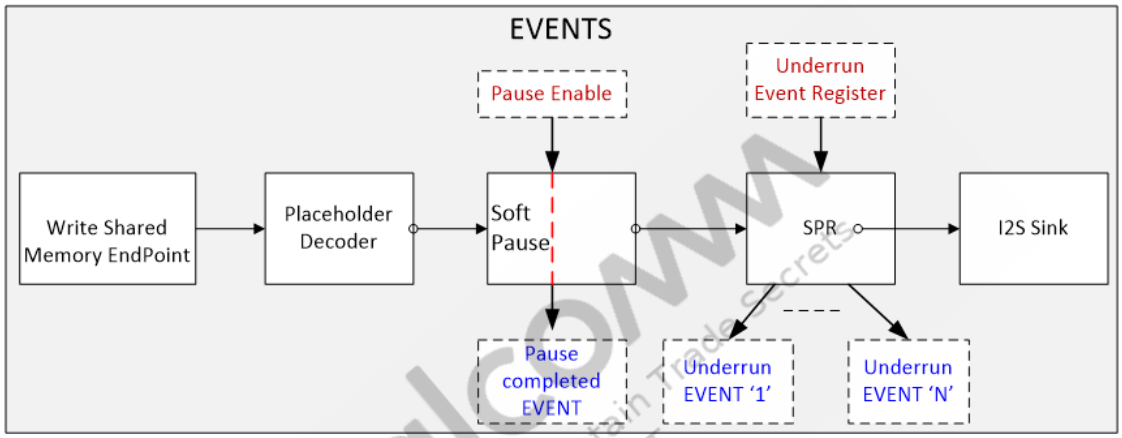
NOTE: In this diagram, although different paths are depicted to describe the metadata types, the propagation happens along with audio data through the same data path interface between the modules in the SPF processing chain.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.2.2.2 Events

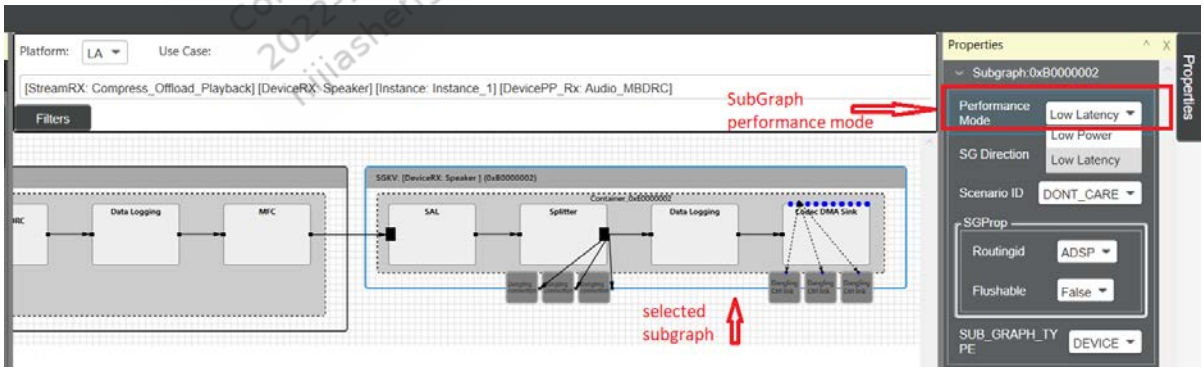
Events include:

- Response events indicating buffer consumption (packet return to HLOS client, watermark level hit in Pull mode), end-of-stream (EoS), pause and resume complete events
- Triggered events like an underrun event reported in the processing chain in the SPF

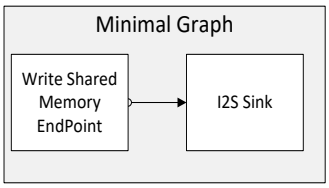


6.2.3 Session control related

- Low power and low latency use cases are supported via configuration of subgraph properties. Flexible graphs (choice of modules and containers) can be created for finer latency control.

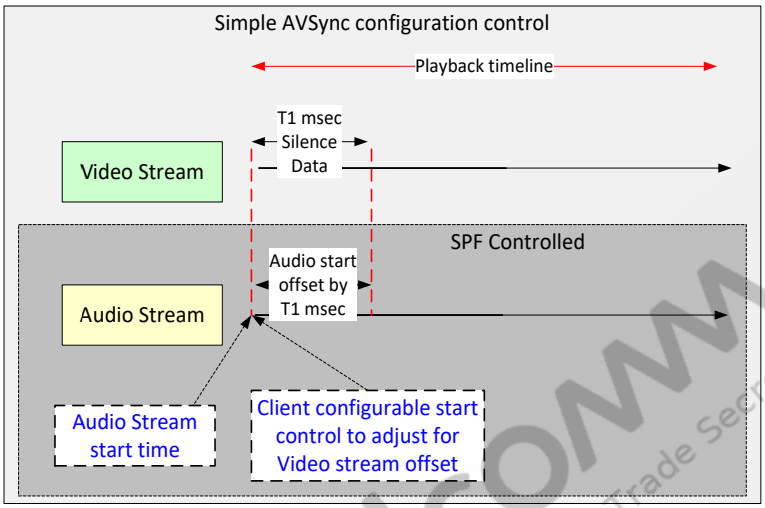


The following is an example SPF graph depicting graph flexibility with minimum modules.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

- Concurrent playback is supported through multiport modules and non-linear graphs for multiple input streams mixing and routing to multiple sink devices.
- Support for audio-video synchronization through control timing of playback. SPF provides hooks to query the session time for the time bar display.

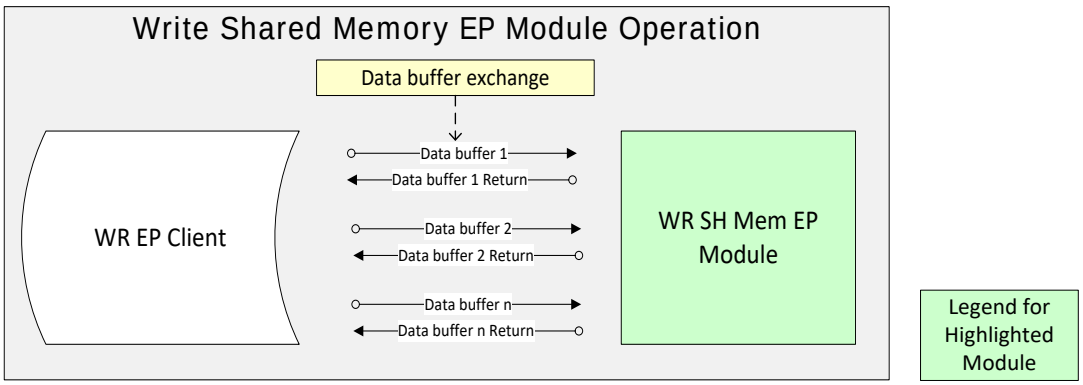


- SPF facilitates rate matching for adjusting playback rate to a sink device's consumption rate through pluggable rate corrector modules and a control link mechanism for consumption rate drift communication.

6.3 Framework modules for playback use cases

6.3.1 WR SH Mem EP

This module is used for the HLOS client data interface in Packet Exchange mode. The HLOS client should map the memory for the data buffers via memory map commands before starting the buffer exchange.



The data buffer payload contains buffer timestamps and flag information. Flags indicate additional information such as timestamp validity, last buffer, end-of-frame, etc. Data buffers are returned via a data command response message.

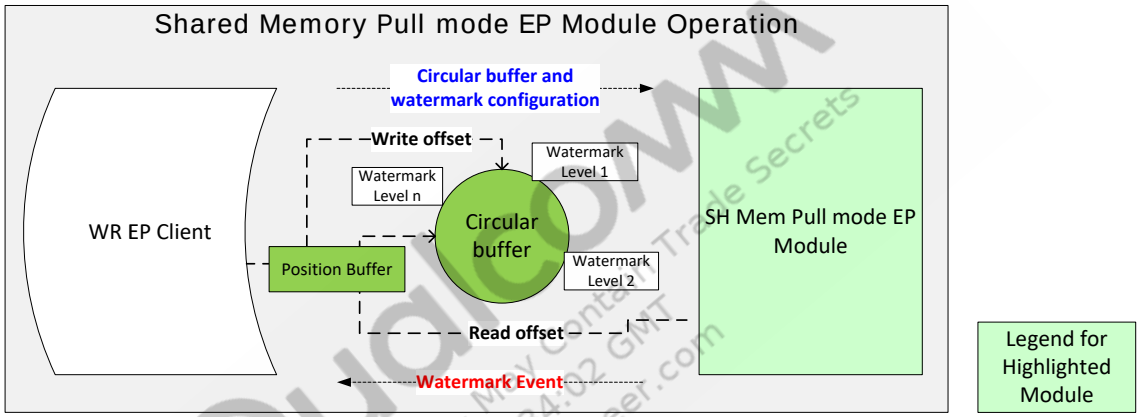
MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The media format for this module is configured by the HLOS client and is propagated through the graph. The media format indicates the PCM data format. The data buffer size should be a multiple of the number of channels.

An EoS event is raised to the HLOS client when it is received by the hardware endpoint sink or when it is consumed/dropped by any module in the graph.

6.3.2 SH Mem Pull Mode EP

This module is used for the HLOS client data interface in Non-Packet Mode. This mode is used in cases where packet exchange can cause issues due to IPC latencies and is advantageous for ultra-low latency (ULL) playback.



With this module, a shared circular buffer is used by the HLOS client to write the data. The SPF also uses this buffer to read the data. The SH Mem Pull mode EP module is configured with a circular buffer address, a position buffer address (used for synchronization across writer and reader), and media format information. The shared position buffer is used for data write/data read synchronization between two processors (the HLOS and ADSP). Alternatively, the HLOS client can register for watermark levels in the shared circular buffer and listen to the notifications from the SH Mem Pull mode EP module when the read offset crosses those configured watermark levels.

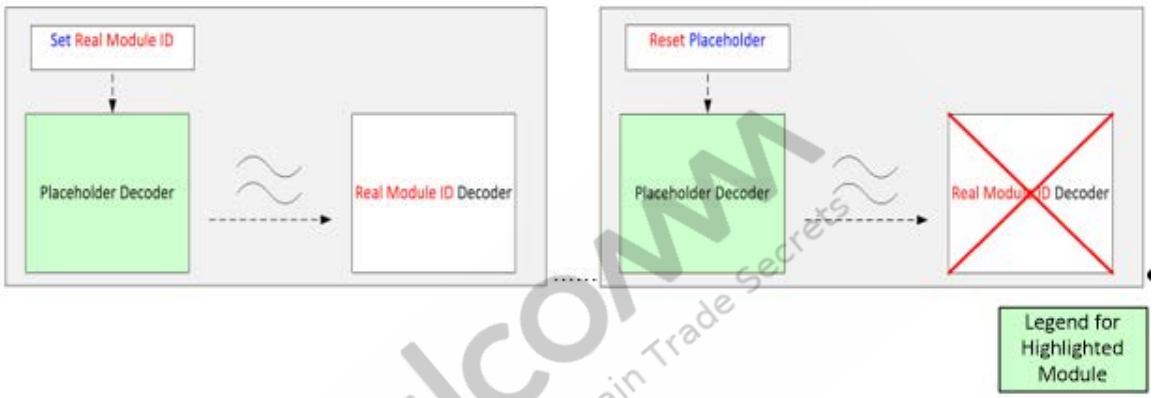
Data path commands such as media format update and EoS are not supported in Pull mode playback once setup is done. Only PCM playback is supported due to the uniform data consumption requirement. Timestamps and metadata propagation are not supported.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.3.3 Placeholder Decoder

This is a placeholder module for the underlying decoder module. It is introduced to keep the graph template uniform for all compressed playback scenarios because having an actual decoder module in the graph would result in per decoder graphs in ACDB and require an ACDB update for new decoder integration.

The actual decoder module ID (such as AAC, MP3, etc.), is sent as a “real module ID” parameter configuration to the placeholder decoder.



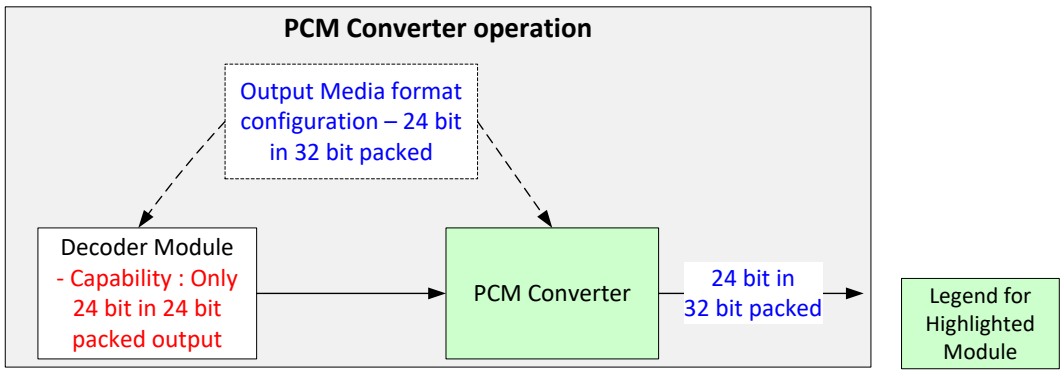
The sequence for placeholder module configuration is as follows:

1. Set enable param.
2. Set real module ID. Any configuration before setting the real module ID is cached and applied after this step.
3. Send module media format.

Qualcomm Confidential – May Contain Trade Secrets 2022-11-05 10:24:02 GMT nijiasheng@longcheer.com

6.3.4 PCM Converter

The PCM Converter module handles data conversions except the resampling operation. The supported operations include interleave/deinterleave, bit width conversion, channel mixing, Q format, and endianness conversions.

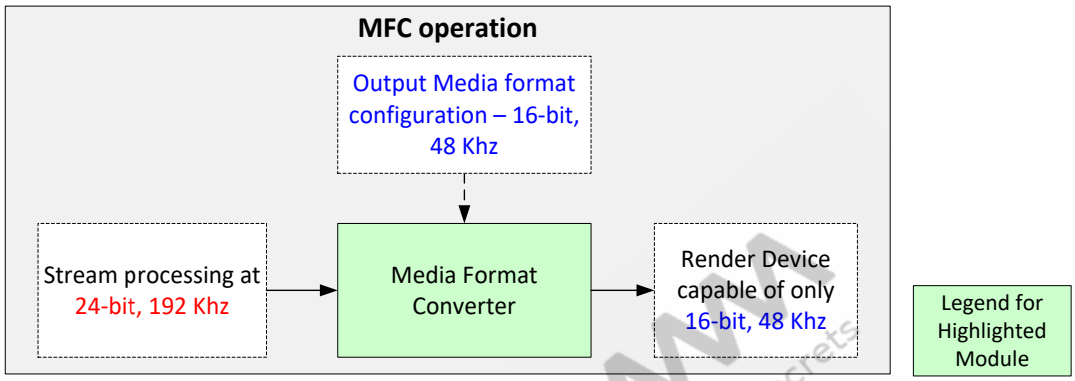


The PCM Converter is placed immediately after the decoder, which helps avoid different types of data conversions across multiple decoder CAPI wrappers. Only the output media format should be configured for the PCM Converter module; the input media format is propagated in the data path.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.3.5 Media Format Converter (MFC)

This module handles channel mixing, sample rate conversion, and bit width conversion For example, the MFC module handles sample rate conversions and stream-to-device conversions such as 24-bit to 16-bit.



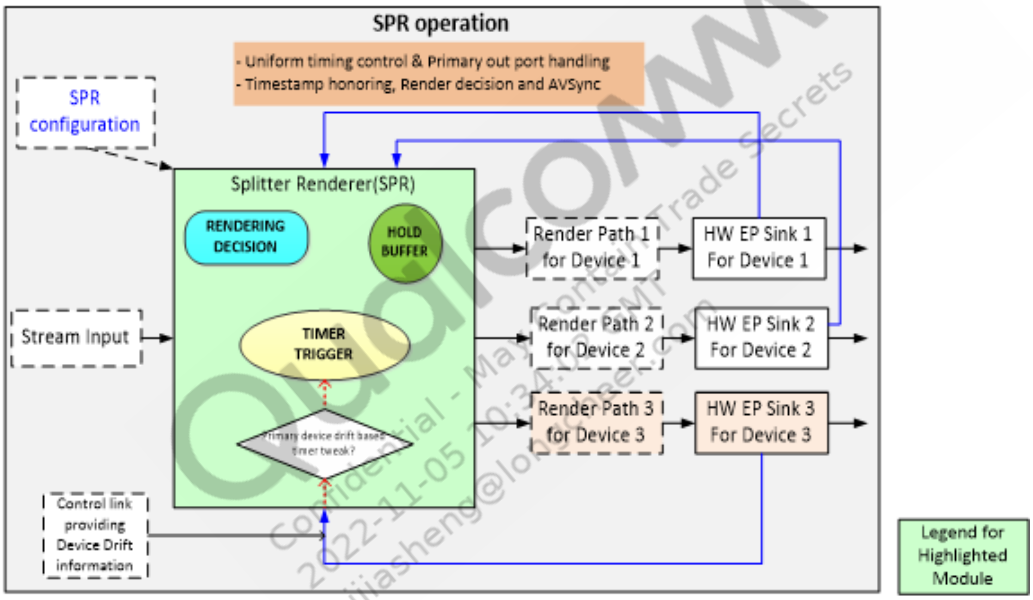
Similar to the PCM Converter module, only the output media format should be configured for the MFC module; the input media format is propagated in the data path.

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT nijiasheng@longcheer.com

6.3.6 Splitter Renderer (SPR)

This module is typically placed at the boundary of the stream subgraph and the device subgraph and is used to split a single stream input to multiple outputs (such as in a SSMD or MSMD scenario). This module perform part of the job done by the Matrix Mixer in the previous Elite framework.

SPR is a signal-triggered module. The timer feature in SPR provides the trigger for uniform timing for rendering data at output ports on each of the timer interrupts. The timer is rate adapted to the hardware endpoint module associated with the primary output port by communicating the drift information via control link connection with the hardware endpoint module. This avoids the need for additional rate matching modules for simple playback use cases.



Rendering decisions in the SPR module are a configurable feature for audio-video synchronization purposes.

The HLOS client can configure the rendering of data stream input to the output ports at the start of playback session in three possible modes:

- Start the rendering immediately (that is, when data at stream input is available)
- Start the rendering with an initial delay configured by a 64-bit delay parameter. This blocks the input port (no buffering of data in SPR module) until the amount of delay time has passed.
- Start the rendering at a specific absolute time

Similarly, the HLOS client can configure the rendering decision during session runtime to be either:

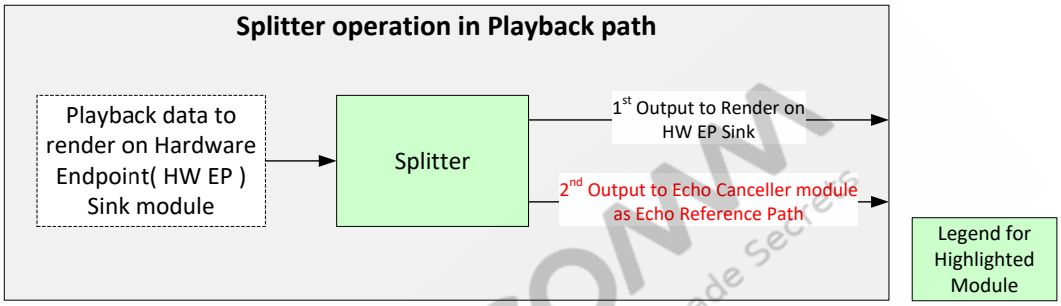
- Default (hardware interrupts generated by the hardware endpoint module are translated to render requests at the SPR module output port)
- Render incoming data with absolute timestamps by comparing with the wall clock time. The SPR module also provides a client-configurable hold buffer for buffering of incoming data in the timestamp mode of operation.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The SPR module reports underrun events in scenarios where data at input is not available upon the trigger of the timer interrupt. The HLOS client can register and receive notifications from the SPR module for underrun events.

6.3.7 Splitter

This module (also called simple splitter) is used to split input data into multiple output data ports. It is used extensively in SPF to realize various configurations across different use cases.

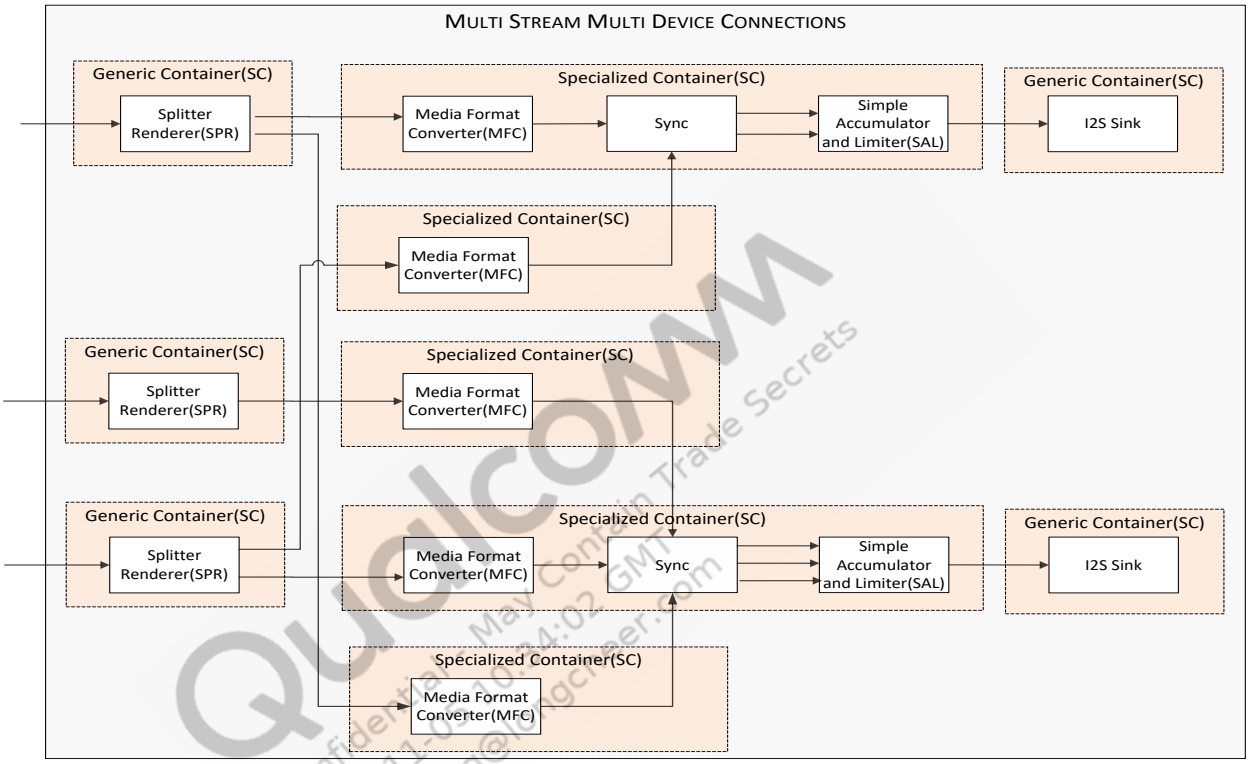


This module is mainly used to split and route playback data into an echo cancellation module to act as an echo reference path. It helps for effective implementation of the music playback with SVA barge-in use case, allowing for minimal configuration in designing the graph in the QACT graph designer too.

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT nijiasheng@longcheer.com

6.3.8 Simple Accumulator and Limiter (SAL)

This module is used to accumulate multiple incoming streams into a single output stream. All data inputs to SAL should be of the same media format and all input ports to SAL should be synchronized by placing the SYNC module before SAL in the graph.

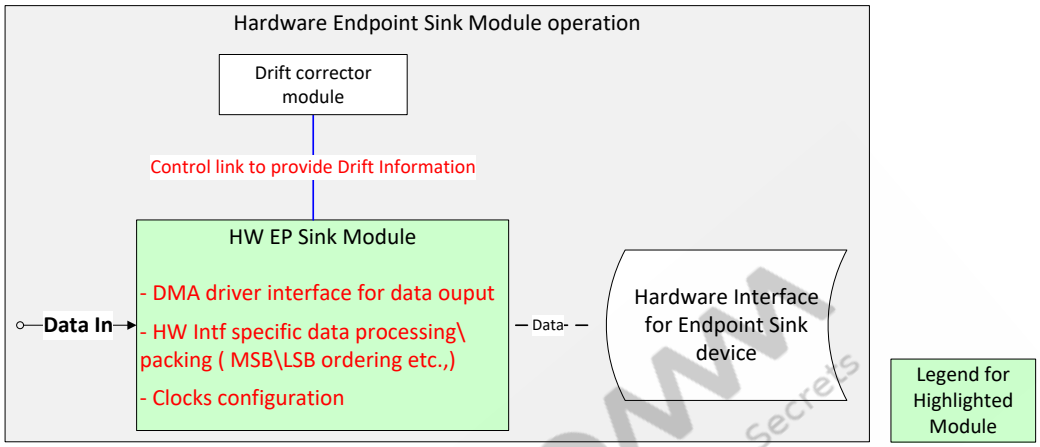


SAL can also limit and modify the bit width of an output signal based on limiter and output format parameter configurations.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.3.9 Hardware Endpoint Sink module

This modules provides abstraction for actual hardware sink interfaces such as I2S, SLIMbus, codec DMA, etc. that act as playback output data transfer endpoints.



The Hardware Endpoint Sink module is a signal-triggered module that is driven by hardware data interrupts. It receives two types of configurations:

- Common configurations such as media format, frame size, and power mode configurations related to LPI and XO shutdown
- Interface-specific configurations such as interface-specific data processing configurations (for example, codec DMA configuration, display port configuration, etc.) and the DMA transfer driver interface.

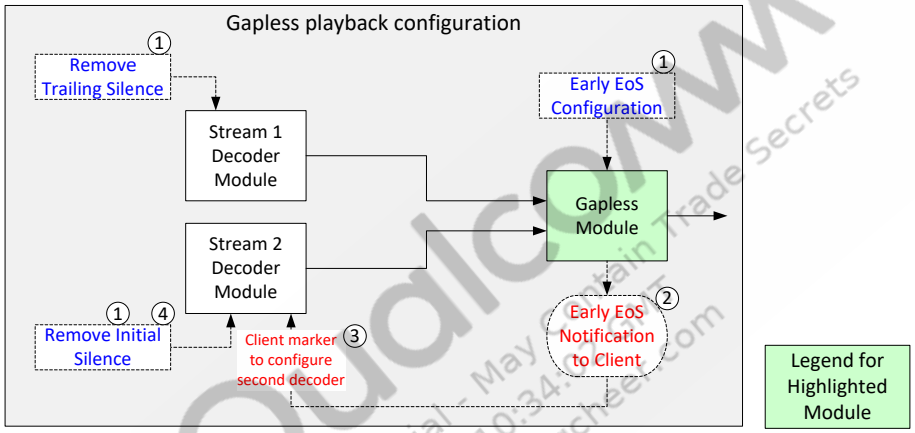
The Hardware Endpoint Sink module provides clock drift information via control link to be used by drift corrector modules in the topology. Also, EoS events in the playback path are generally raised to the client by the Hardware Endpoint Sink module.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

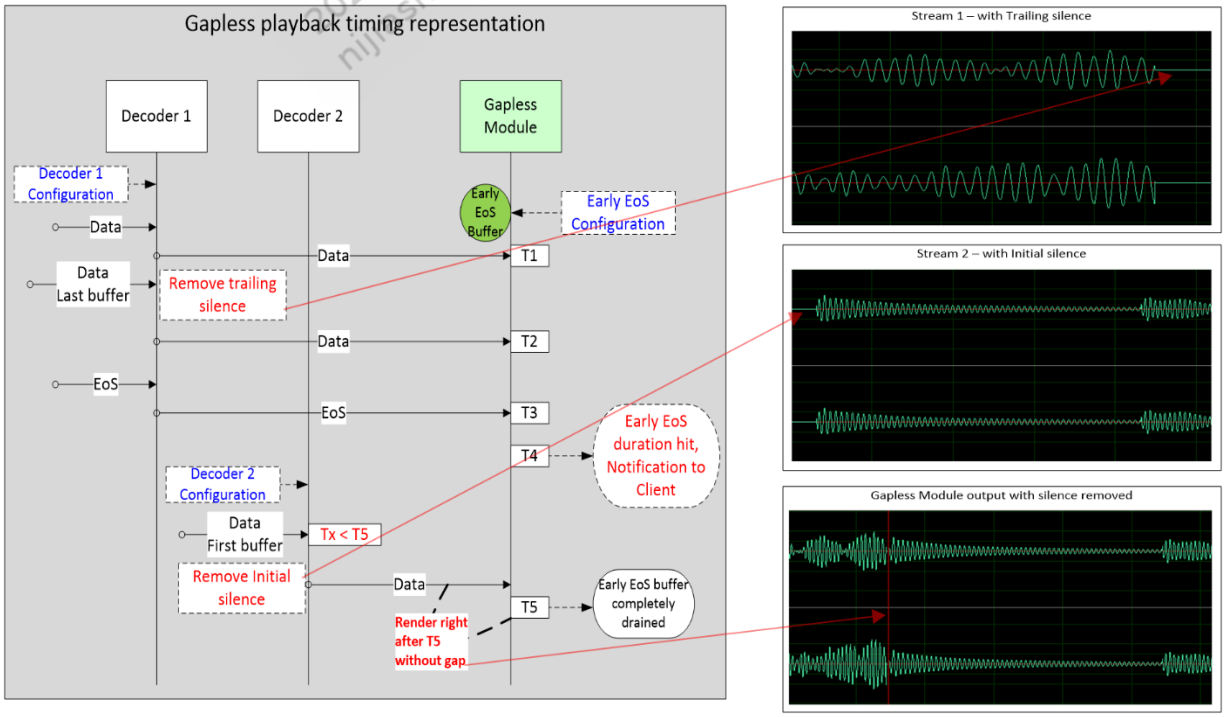
6.3.10 Gapless module

This module is used along with decoder module configurations to realize gapless playback of continuous playback content that are mastered as multiple tracks (also called gapless tracks) where it is expected to be rendered without gaps/silence across the tracks when such tracks are played in a playlist.

The following diagram shows playback operations of the Gapless module with the sequence of commands numbered. Multiple numbers indicate that either sequence is permitted. The configuration shown in the diagram represents the Dual Stream mode of operation. The Single Stream mode of operation has a single decoder and one input port to the Gapless module. An early EoS notification is used to configure the second stream/decoder.



SPF supports gapless playback across different codec formats. The decoder module is configured for initial and trailing silence removal. The Gapless module is configured for early EoS notification with a required early EoS duration.

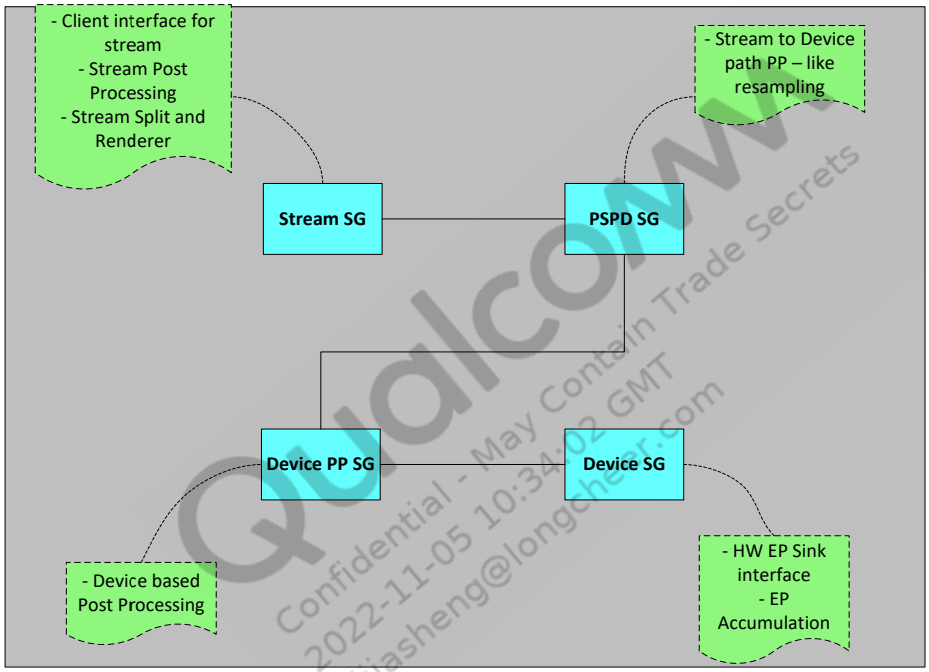


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.4 Playback operations

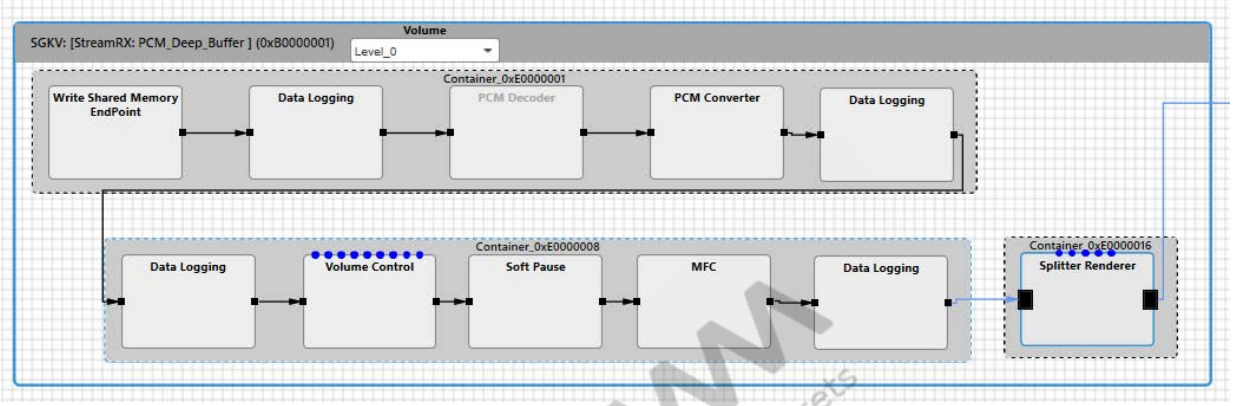
6.4.1 Typical playback graph

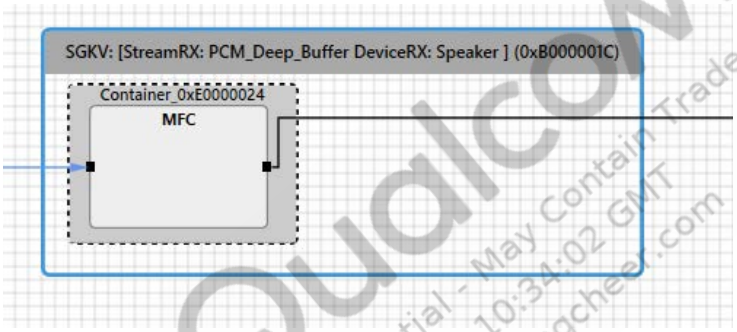
SPF facilitates implementation of subgraph properties to cater to latency requirements of use cases and support use case handling via graph management commands addressed to subgraphs. The use case graph is broadly divided into four subgraphs (stream subgraph, per-stream per-device subgraph, device postprocessing subgraph, and device subgraph.

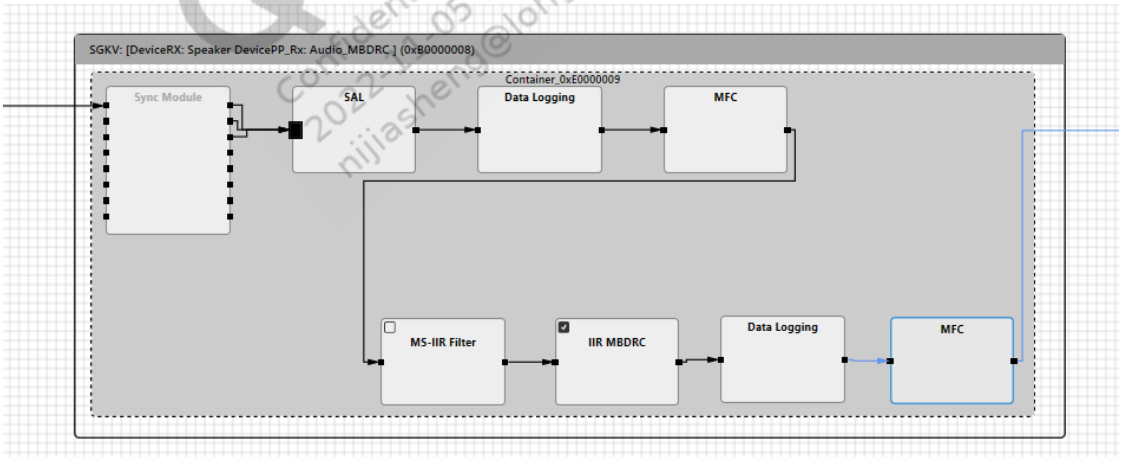


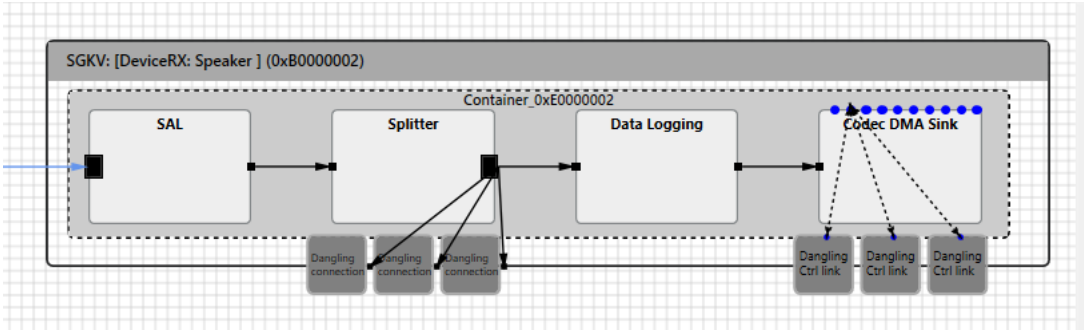
Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT nijiasheng@longcheer.com

The following diagrams show each of the subgraph types, modules, container grouping, data links, and control links in a typical playback path graph in SPF. The most common capabilities are provided as framework modules in SPF while other signal processing modules can be customized by OEMs as per their needs.









Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT nijiashengelongcheer.com

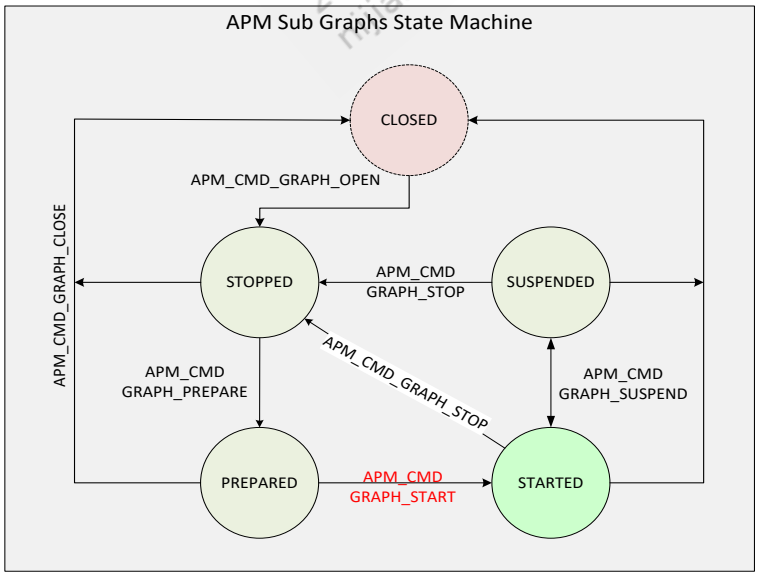
The salient points for audio playback in SPF are as follows:

- Hardware endpoint modules should receive the mandatory media format and interface configurations
- The output media format (that is, sample rate, bit width, channels) of the MFC module in the device leg should be configured so that it matches the hardware endpoint module configuration
- The placeholder decoder should receive real_module_id before the graph START command. The real ID cannot be changed at runtime unless the subgraph is STOPPED.
- The media format is propagated through each module at the graph PREPARE and START commands
- The client can queue data buffers before the START command, but SPF will process and return buffers only after the START command
- The client sends an EoS marker with the last buffer in a data path. EoS is propagated through the entire topology and the EoS rendered event is raised to the client once the marker reaches the hardware endpoint module. After EoS rendered ACK, the client sends the GRAPH_STOP command.

6.4.2 Start/run

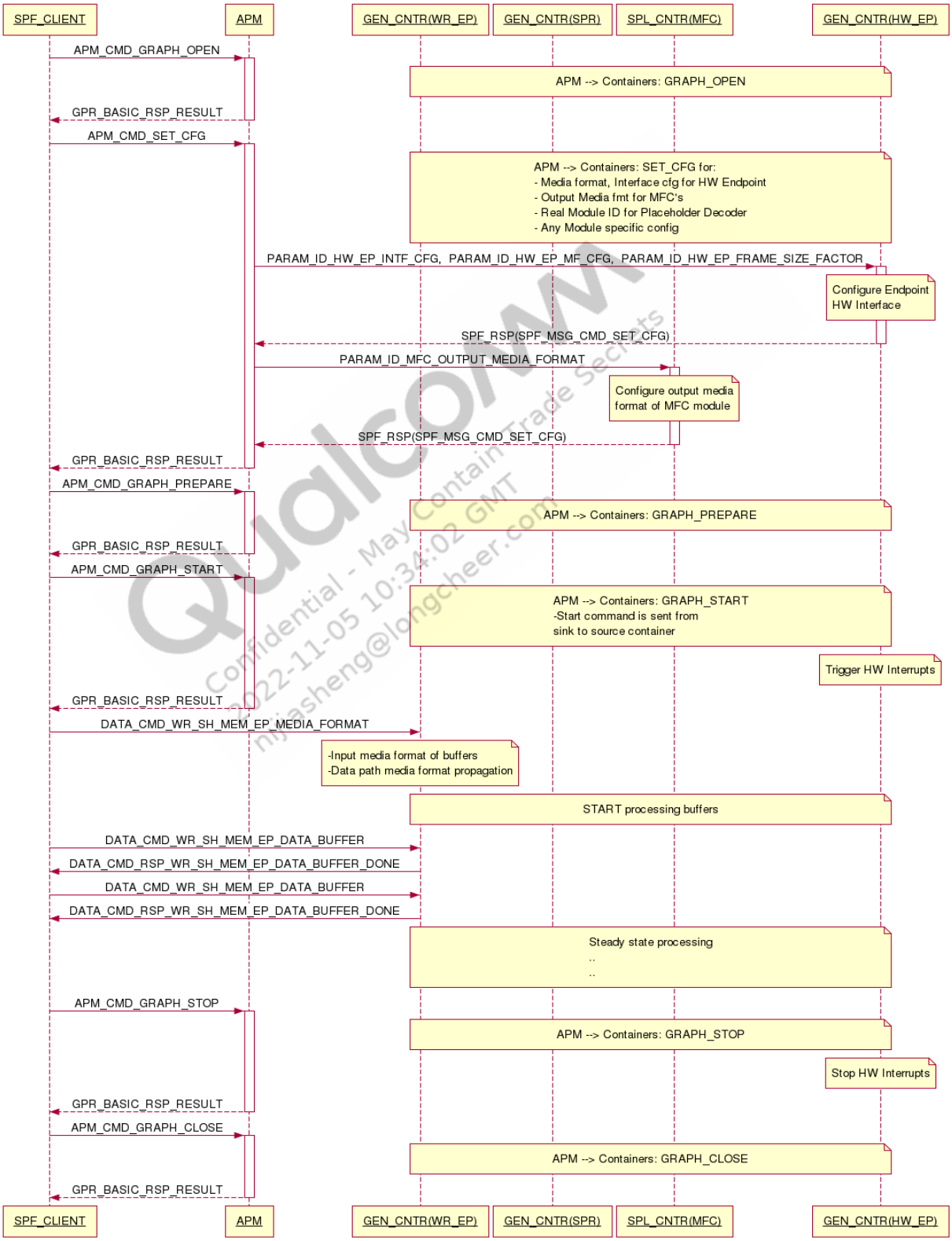
The graph START command is used to start subgraphs. It indicates that data flow has started and that the media format is propagated in the data path. All modules in the graph place their votes for resources such as MPPS, bus bandwidth, and sleep latency upon receiving this command. It is also used to indicate a Resume after a Suspend state.

The following diagram shows the subgraph state machine in SPF.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The following diagram represents the call flow for an audio playback session indicating the sequence of graph commands sent from the HLOS client to the SPF APM module and the corresponding ACKs received.



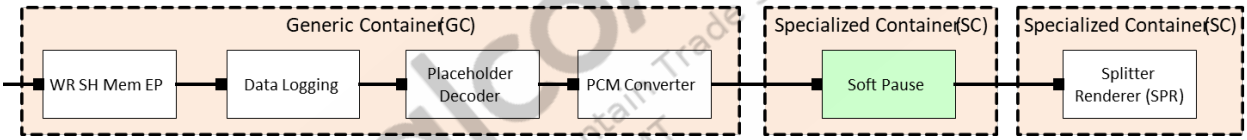
Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT mjiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.4.3 Pause, seek, and resume

The pause operation is handled via the Soft Pause module placement in the graph that needs to be paused. The Soft Pause module configuration options are:

- Pause start – Indicates to pause data flow downstream
- Resume – Indicates to start data flow downstream
- Ramp parameters that can be configured:
  - Ramp duration
  - Ramp step size
  - Ramp curve: – Linear, exponential, or logarithmic
  - Downstream delay
  - Deferred ramp up
  - Ramp threshold



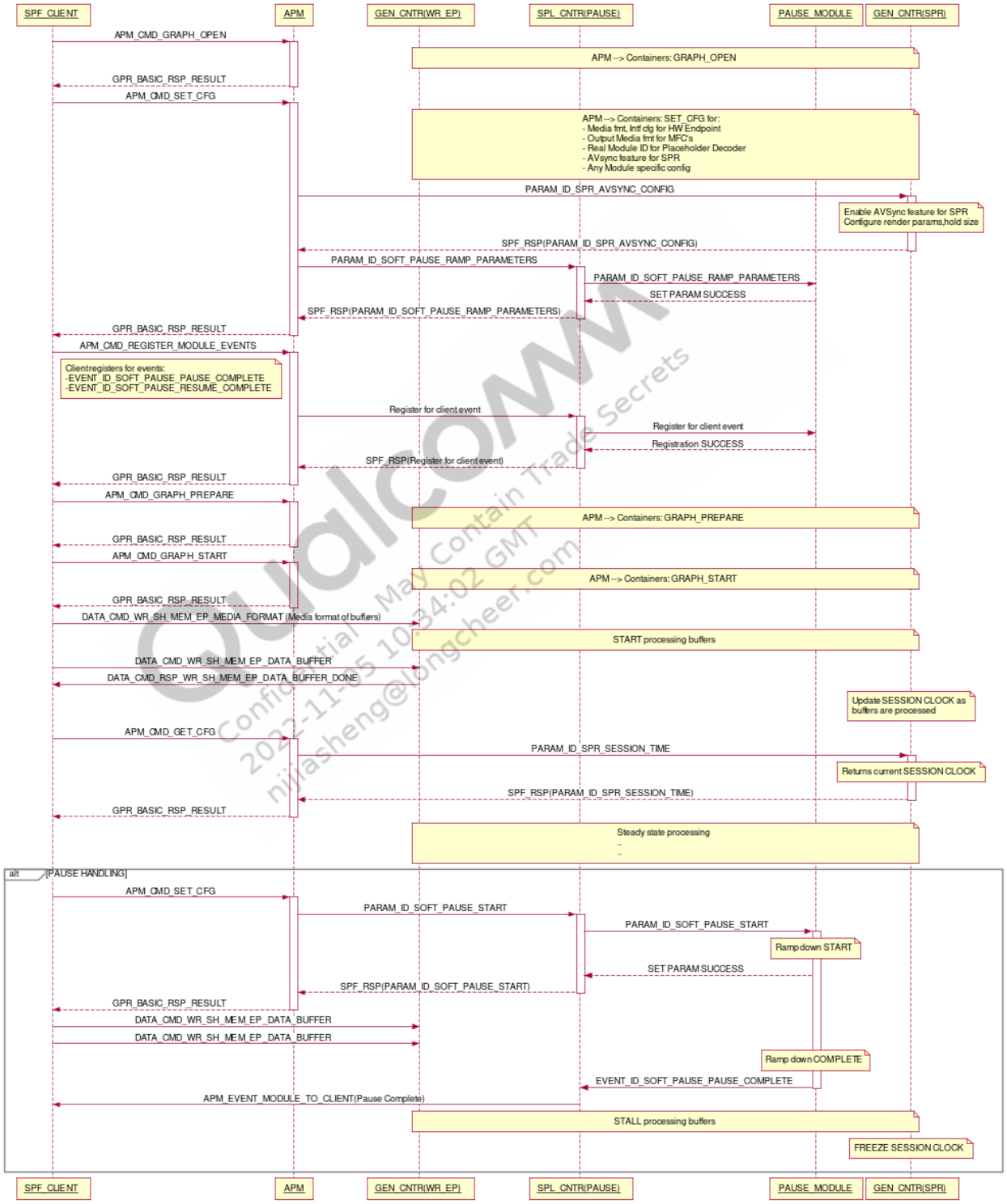
The Flush command is issued to handle the seek operation during playback. When a user performs a fast forward or rewind in the playback UI, it is expected to flush all the existing data in the playback topology before handling new data that is discontinuous with regards to the current data being played.

The salient points for pause, seek, and resume are as follows:

- The AV sync feature should be enabled and configured through set cfg for SPR
  - Configure render mode (immediate, delayed), render window, and hold duration
  - Set cfg should be done before START; cannot be enabled at runtime
  - If AV sync is not enabled, session time is returned as 0
  - Session time query is done through get cfg to the SPR module
  - If there is a STOP -> START transition, SPR will proceed with the previous AV sync configuration unless otherwise configured in the Stop state
- Pause module ramp parameters should be configured for smooth ramp up/down. Default ramp parameters are 0.
- PAUSE-RESUME and STOP-START are decoupled. That is, if a subgraph is in Stop state and the pause module receives RESUME, it will not begin ramping up/resuming playback until START is issued and data buffers are available.
- FLUSH is used to reset the session time and return buffers for the seek operation. The same functionality can be obtained through the START-STOP sequence.
- If the graph is flushed before pause ramp down is completed, glitches could occur. This can be avoided by registering for the pause complete client event and performing a flush after notification

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

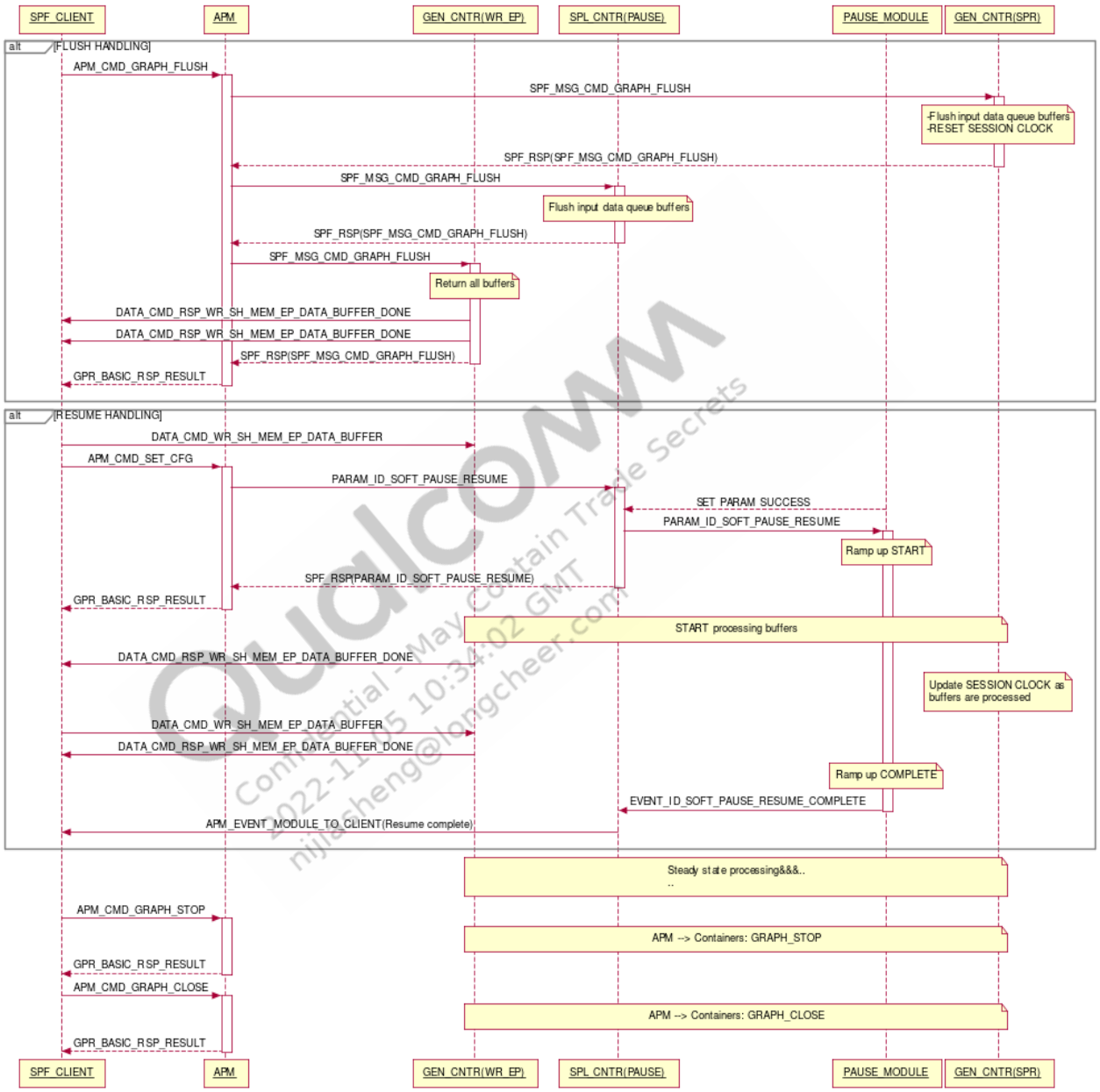
The following call flow diagram provides the sequence of commands involved in playback pause handling.



Qualcomm Confidential May Contain Trade Secrets 2022-11-15 10:34:02 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

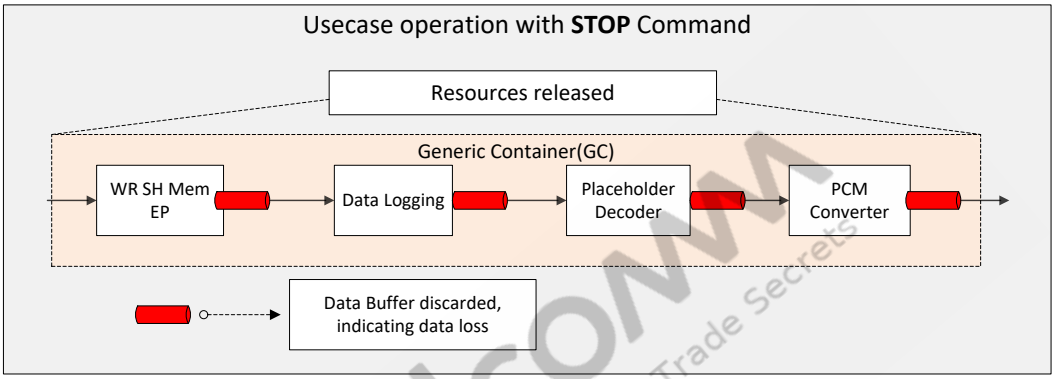
The following call flow diagram provides the sequence of commands involved in the flush and resume operation.

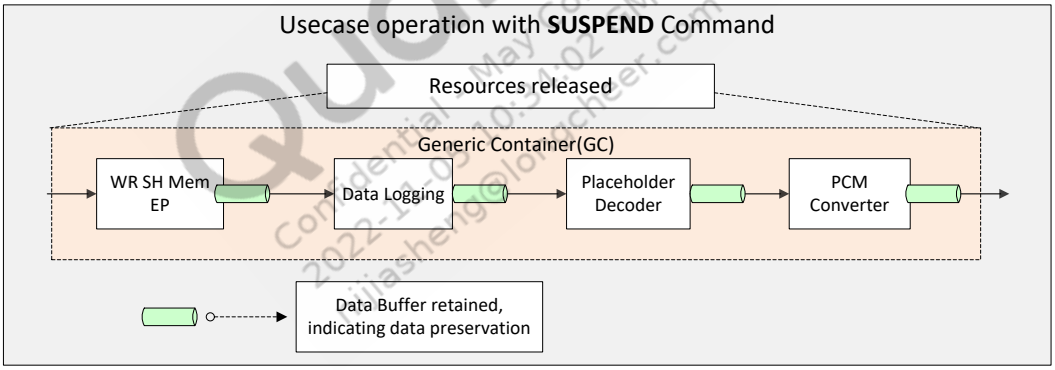


Qualcomm Confidential - May Contain Trade Secrets 2022-11-15 10:34:02 GMT nijiacheng@longcheer.com

6.4.4 Suspend

The graph SUSPEND command is used for scenarios such as long pause where the session state with data needs to remain intact while just relinquishing all hardware resources. It is preferable to SUSPEND in a long pause scenario instead of STOP to ensure that the data loss associated with STOP-START sequence is avoided and the power benefits of a STOP are retained. The START command is issued to resume from the Suspend state.



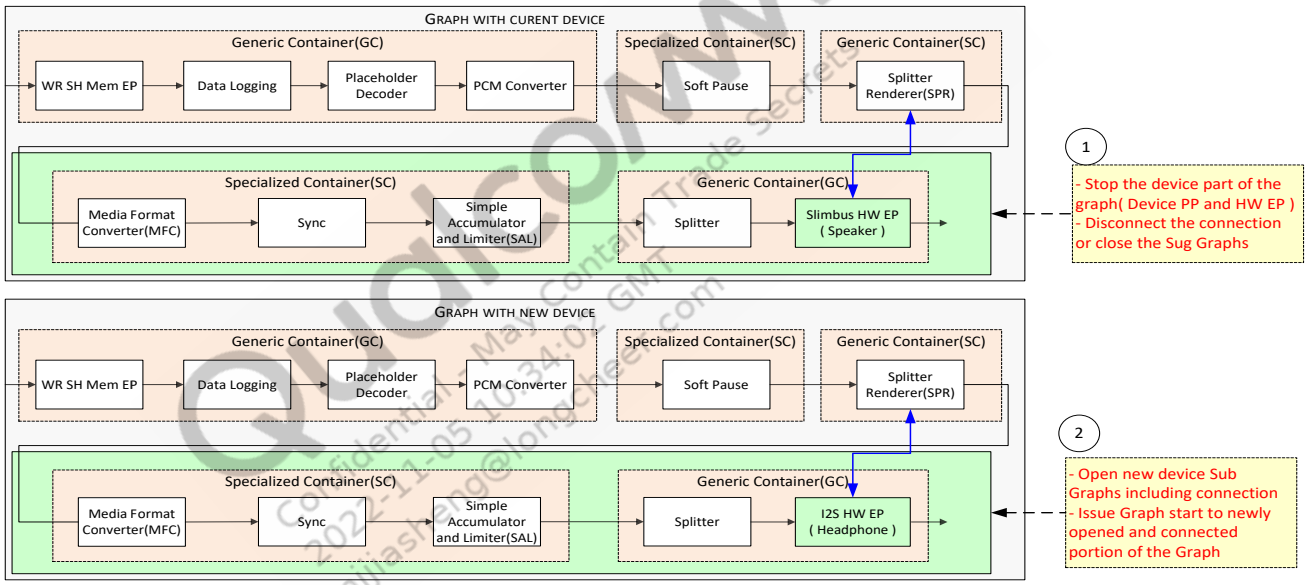


Qualcomm Confidential – May Contain Trade Secrets 2022-11-03 10:31:02 GMT lijia-heng@lorwacher.com

6.4.5 Device switch

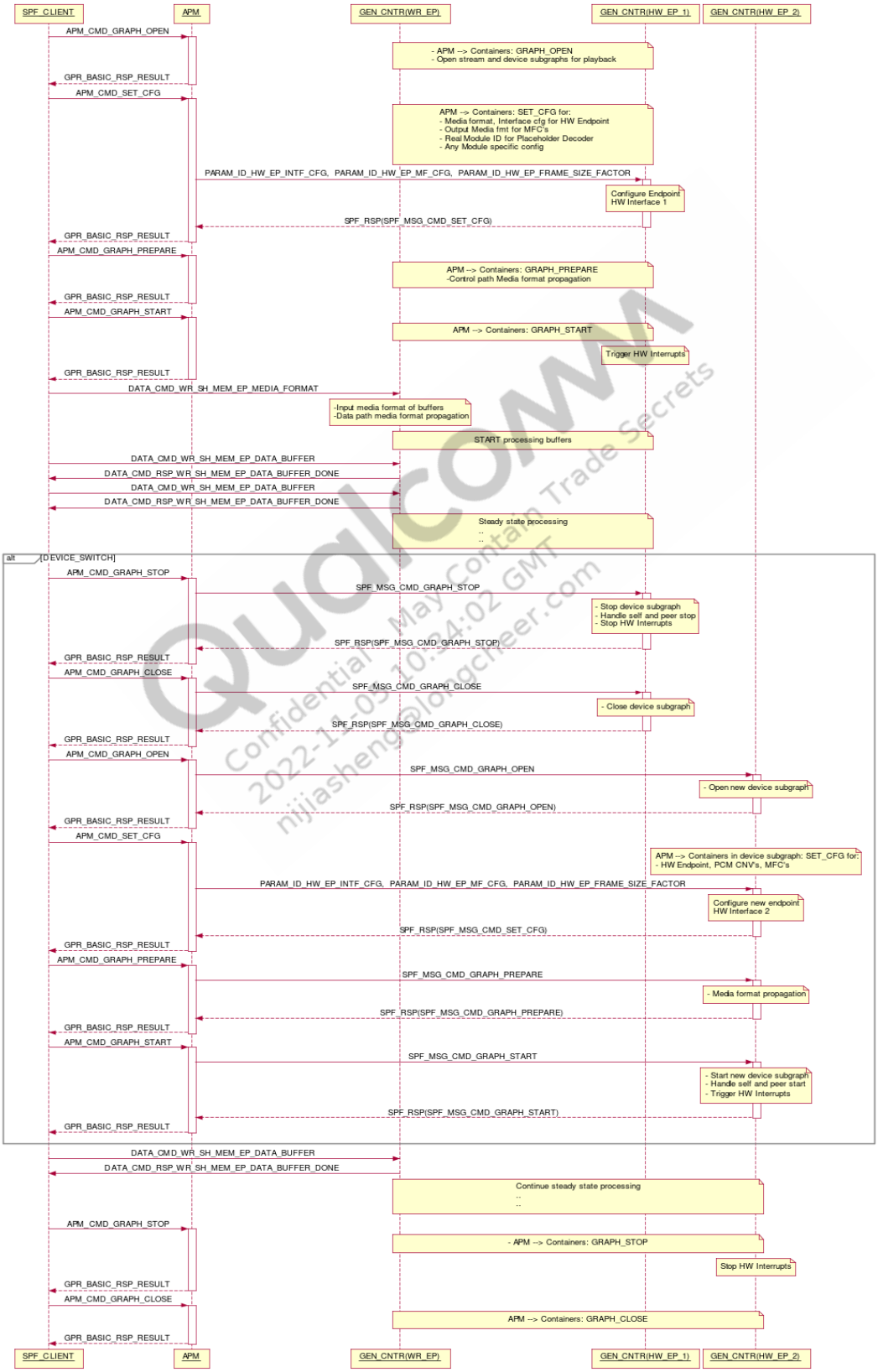
The salient points for device switch are as follows:

- Translates to STOP -> CLOSE -> OPEN -> CONFIGURE -> START for the device subgraph
- When the device subgraph is stopped during switch:
  - SPR gets peer STOP and disable timer
  - Downstream stops consuming data and WR buffers will not be returned to the client
- The hardware endpoint media format and interface configuration need to be sent again for the endpoint module in the newly opened device subgraph



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The following call flow diagram shows the sequence of commands involved in a device switch scenario during the playback use case.



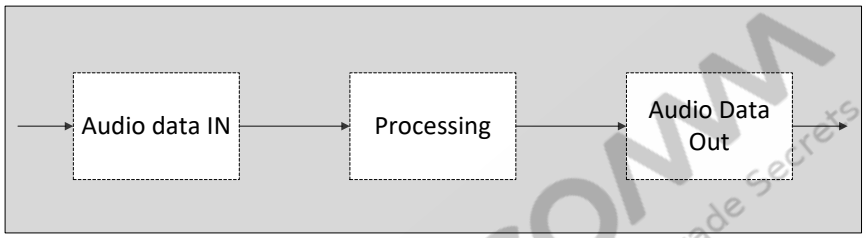
Qualcomm Confidential May Contain Trade Secrets 2022-11-03 10:34:02 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

7 SPF audio record

7.1 Overview

The audio record in the SPF can be generally represented by the following diagram consisting of three logical blocks (similar to audio playback).



7.1.1 Audio data IN

This logical block represents module(s) handling audio data input devices such as source devices. Examples are modules that interface with source devices such as I2S, SLIMbus, codec DMA, TDM etc.

7.1.2 Processing

This logical block represents module(s) that handle audio processing operations. For example, preprocessing modules such as MFCs, ECNS, volume control, etc.

7.1.3 Audio data OUT

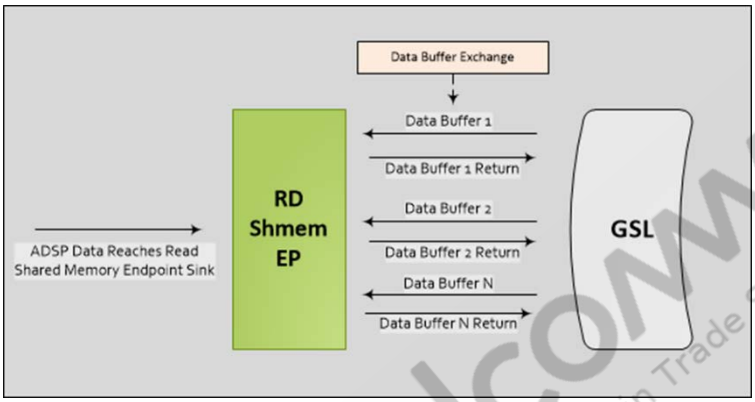
This logical block represents module(s) handling audio data input via Packet Exchange mode or shared circular/ring buffers (no packet exchange). Example of this type of module are the read shared memory endpoint (RD SH Mem EP) module and Push mode EP module.

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:06 GMT nlyaheng@longcheer.com

7.2 Framework modules for record use cases

7.2.1 RD SH Mem EP

This endpoint module is used for the HLOS client interface during Packet Exchange mode. It is supported in generic containers only. The HLOS client directly addresses the RD SH Mem module instance ID.

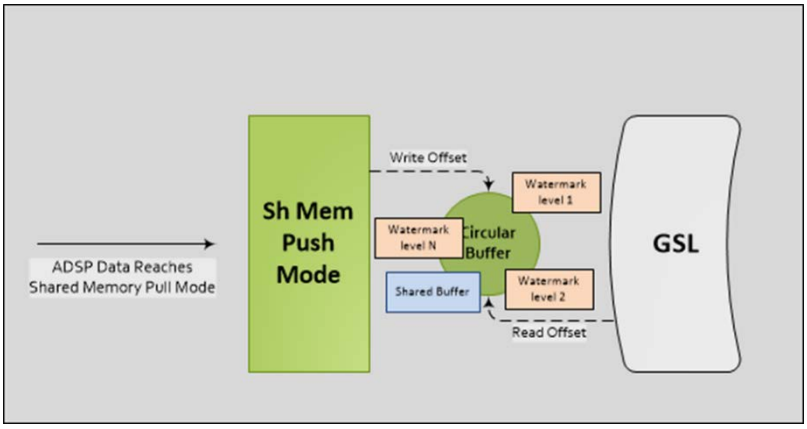


The HLOS client allocates and maps memory for data buffers via memory map commands before starting buffer exchange. The buffers are returned via a data command response message.

The data buffer payload has buffer timestamps and flags indicating additional information such as timestamp validity, last buffer, end-of-frame, etc. The module sends an event to the client to inform it of the media format (the client must register for this event).

7.2.2 SH Mem Push mode

This endpoint module is used for the HLOS client data interface in Non-Packet mode where packet exchange can cause issues due to IPC latencies. It is supported in generic containers and specialized containers.

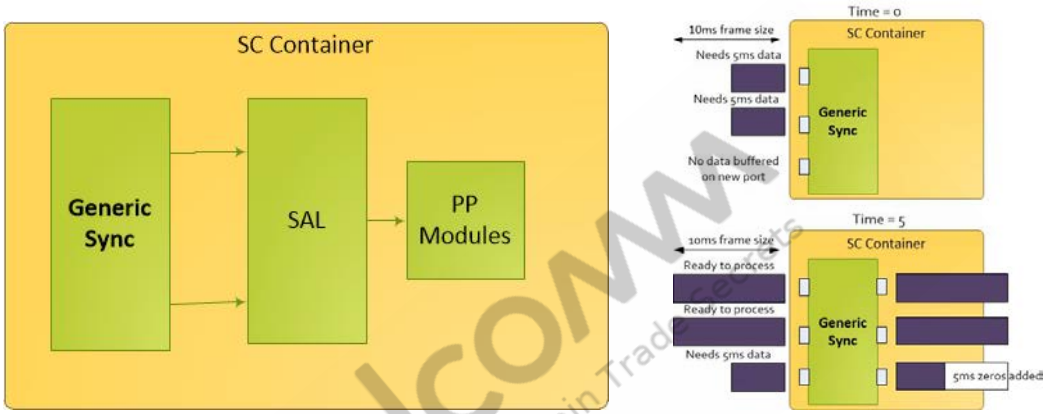


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

A shared circular buffer is used for the client to read data and by the SPF to write the data (supporting only PCM data). EoS and media format data messages are not supported. Timestamps and metadata are not supported.

7.2.3 Generic Synchronization

This module synchronizes data from multiple input streams. It is supported only in specialized containers.



This module should be placed before the SAL module in the following scenarios:

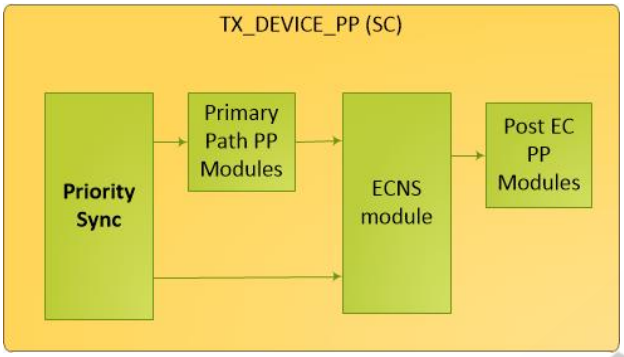
- Incoming data messages from different input ports have different sizes, or
- Incoming data messages have a different size than the container's frame size
- If the SAL module is in a signal-triggered container, then the Generic Sync module is not needed.

Input ports can be opened, started, stopped, and closed while other streams are running. Currently running streams are not delayed or stalled when a new stream is joined as zeros are inserted initially on the new stream when other streams have enough data for processing. For real-time input, input ports are synchronized based on time of arrival.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

7.2.4 Priority Synchronization

This module is needed before any ECNS module (including custom ECNS modules) to synchronize data between the input ports. This module is supported only in specialized containers.



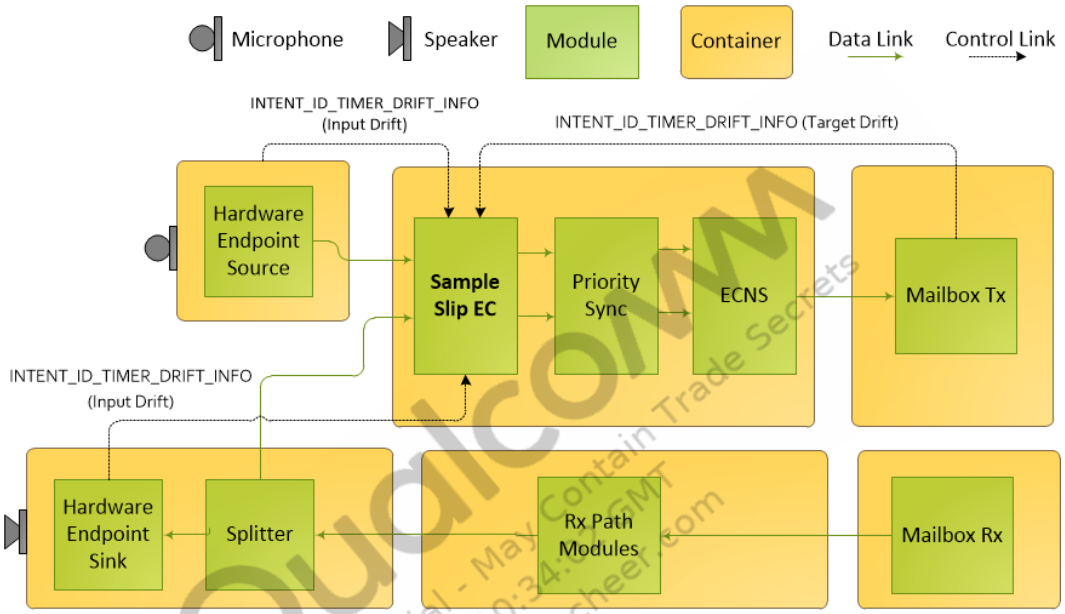
In this module, input ports are prioritized (primary = mic data, secondary = reference data). Incoming data is held until a full frame of primary data is buffered. If the secondary path starts first, data coming before the primary starts is dropped. If the secondary path starts later, zeros are padded to secondary port output until a secondary port starts. If the secondary path data is not present, zeros are inserted on the secondary path, which can help debugging quality issues in ECNS use cases.

Data logging modules can be placed downstream of the priority sync module to check synchronization of the two streams. EoS is destroyed on the secondary path since the module generates secondary data even after the stream ends.

Qualcomm Confidential – May Contain Trade Secrets 2022-11-03 06:02:26 nijiasheng@lomuc-sr-r-com

7.2.5 Multiport Sample Slip

This module has multiple input ports which can come from different clock sources and corrects their drift to a target clock source. Input to output is 1:1 mapped where all output ports have drift corrected to the same source. The control ports allow for drift information to be passed to the sample slip module from input sources and from a target (output) source.



This multiport module approach allows for samples from different ports to be slipped/stuffed at the same position in time (currently the same frame, not the same sample), leading to less artifacts and higher detection rates for voice user interface (VUI) use cases.

7.2.6 Placeholder Encoder

As indicated by the module name, this is a placeholder module for the underlying encoder module. It is introduced to keep the graph template uniform for all types of record scenarios because having an actual encoder module in the graph would result in per encoder graphs in ACDB and require an ACDB update for new encoder integration.

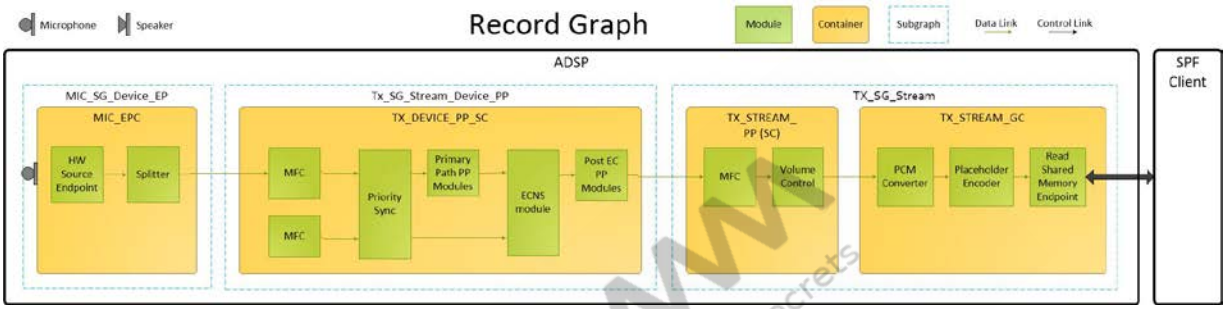
The actual encoder module ID (such as AMR, AAC, etc.) is sent to the placeholder encoder as a real module ID parameter configuration.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

7.3 Use cases

7.3.1 Basic record

The following diagram depicts modules, container groupings, data links, and control links in a typical record path graph in the SPF.

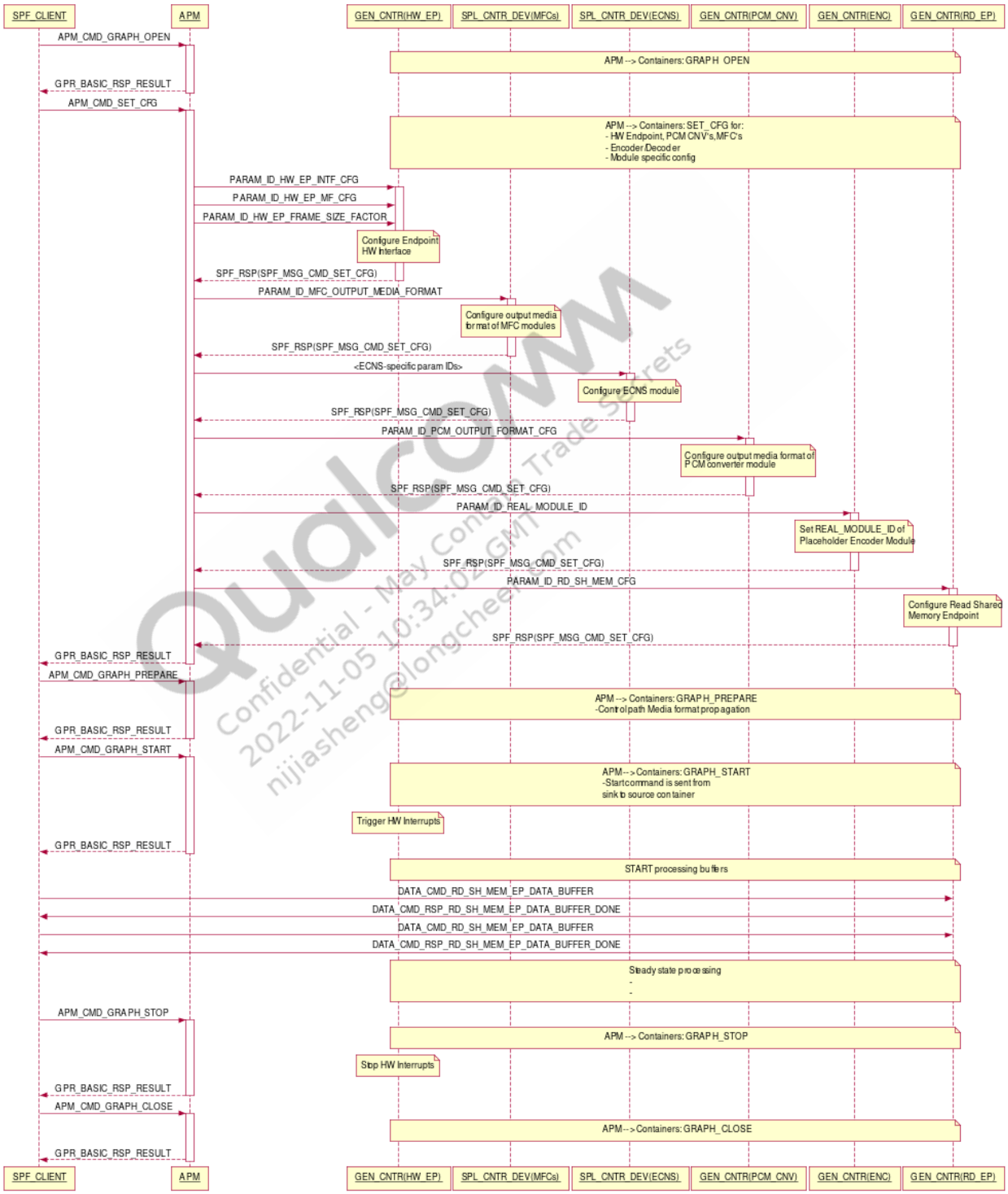


The salient points for an audio record path graph are as follows:

- The PCM converter is to be placed before the encoder in the record path
- ECNS with reference path handles playback concurrency (for example, touch tones)
- MFC modules in the device PP subgraph adjust the media format to fit the ECNS module's requirements
- The Priority Synchronization module is required before any ECNS module to synchronize primary path and reference path audio data
- The MFC module in the stream PP adjusts the media format to match to the client's expected media format
- The Placeholder Encoder module allows the same graph to be used with different encoders. The actual encoder module is determined during setup-time configuration.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The following call flow diagram depicts the sequence of commands involved in an audio record session in SPF.

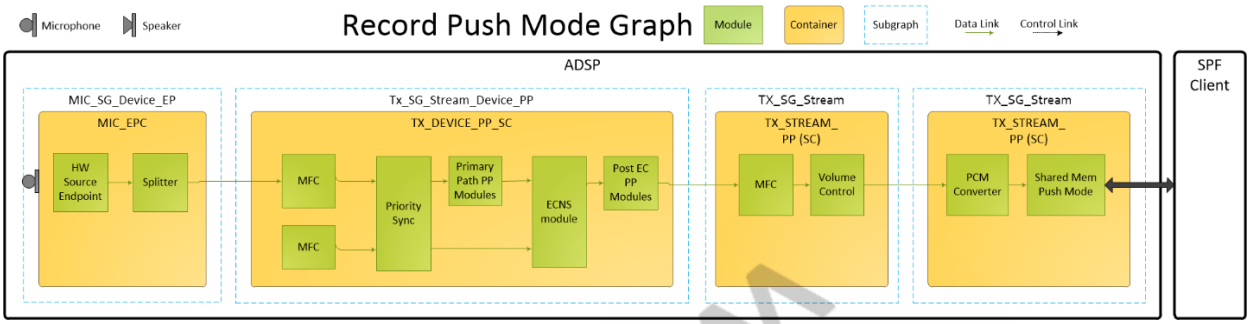


Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

7.3.2 Record with Push Mode EP module

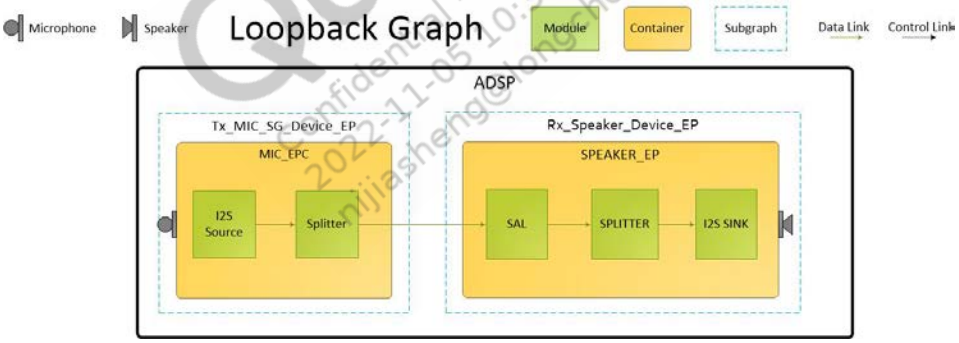
The graph shape for this use case is similar to the read shared memory use case graph except the Pull Mode EP module is used.



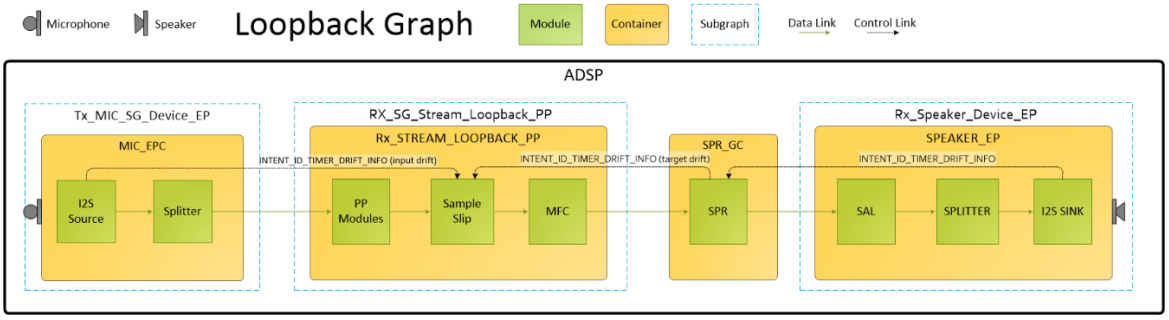
7.3.3 Loopback

Different loopbacks are achievable based on the graph shape in between hardware endpoints. For simple loopback (equivalent to AFE loopback in the Elite framework), source and sink hardware endpoints must come from the same clock source; otherwise, a rate matching module is needed to correct drift.

The following is a simple loopback graph where the source and sink endpoints share the same clock source.

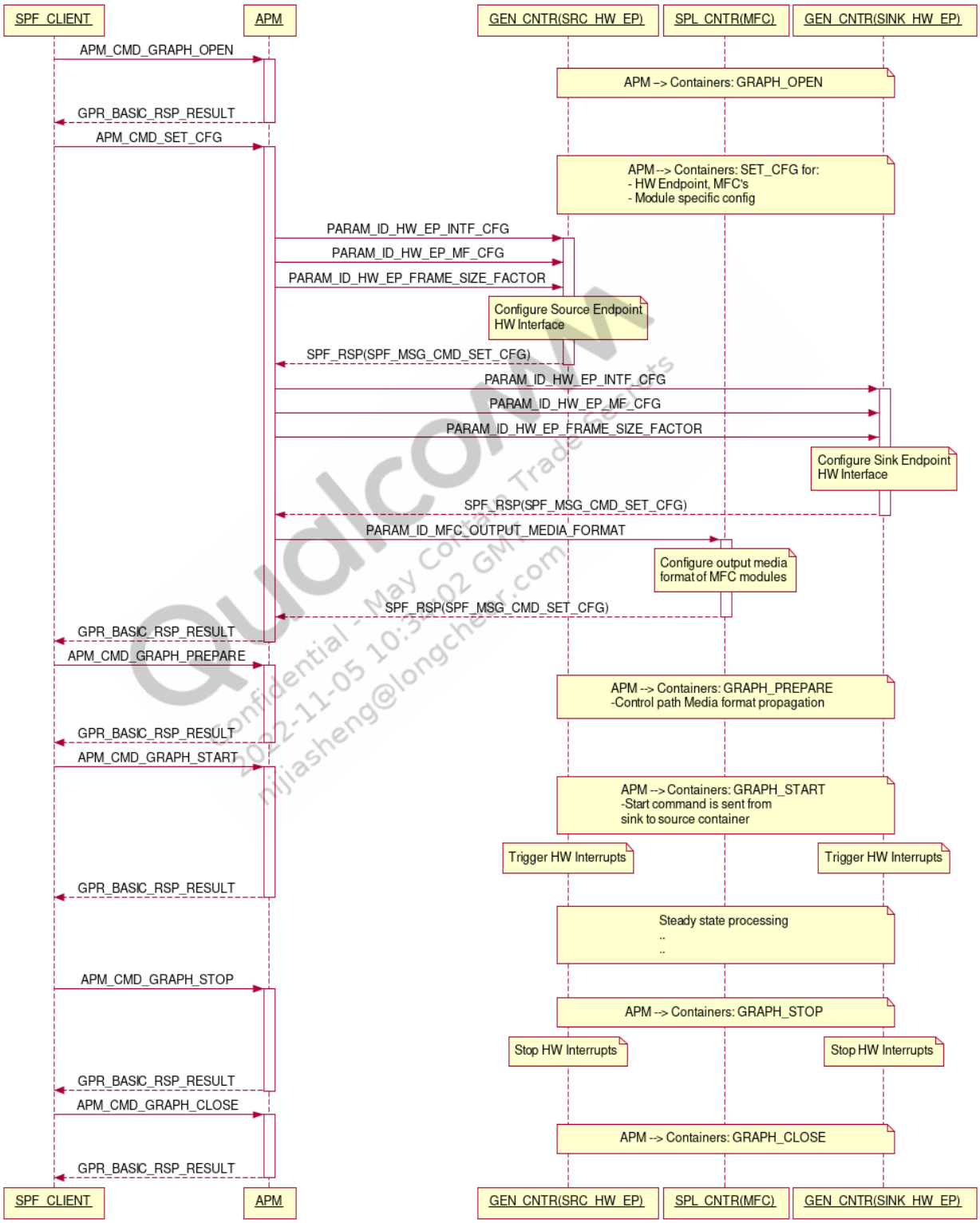


The following is a loopback graph where the source and sink endpoints have different clock sources.



Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT nijiasheng@longcheer.com

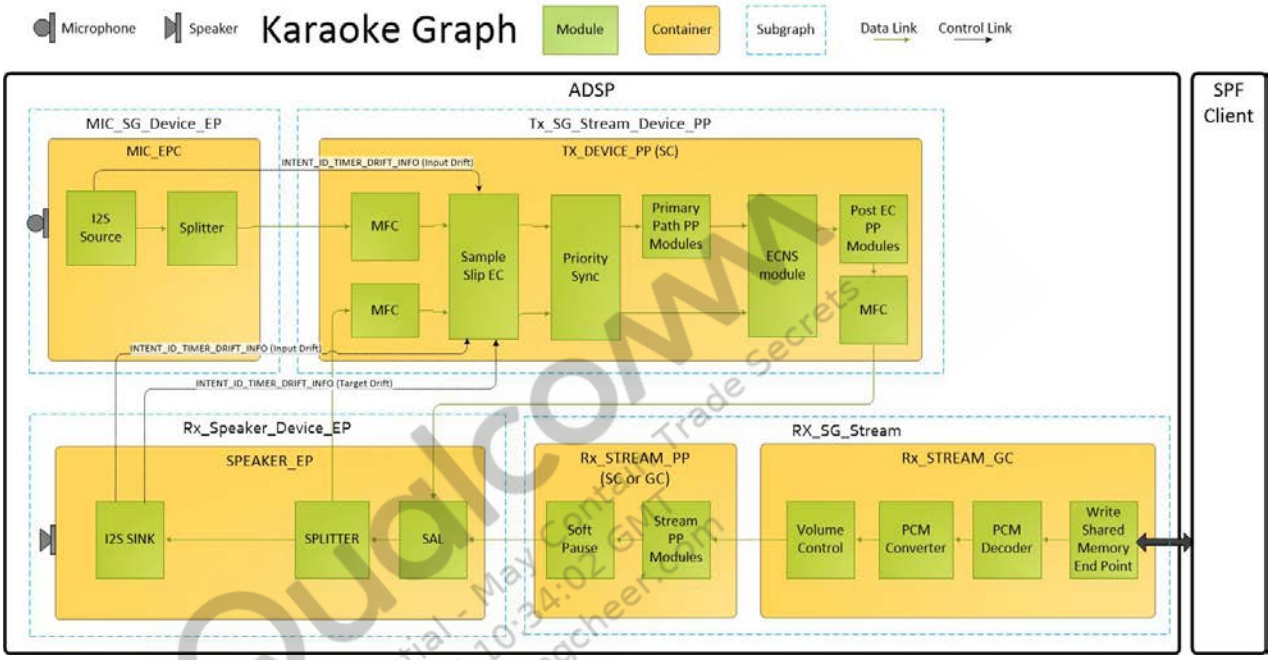
The following call flow diagram depicts the sequence of commands involved in loopback.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

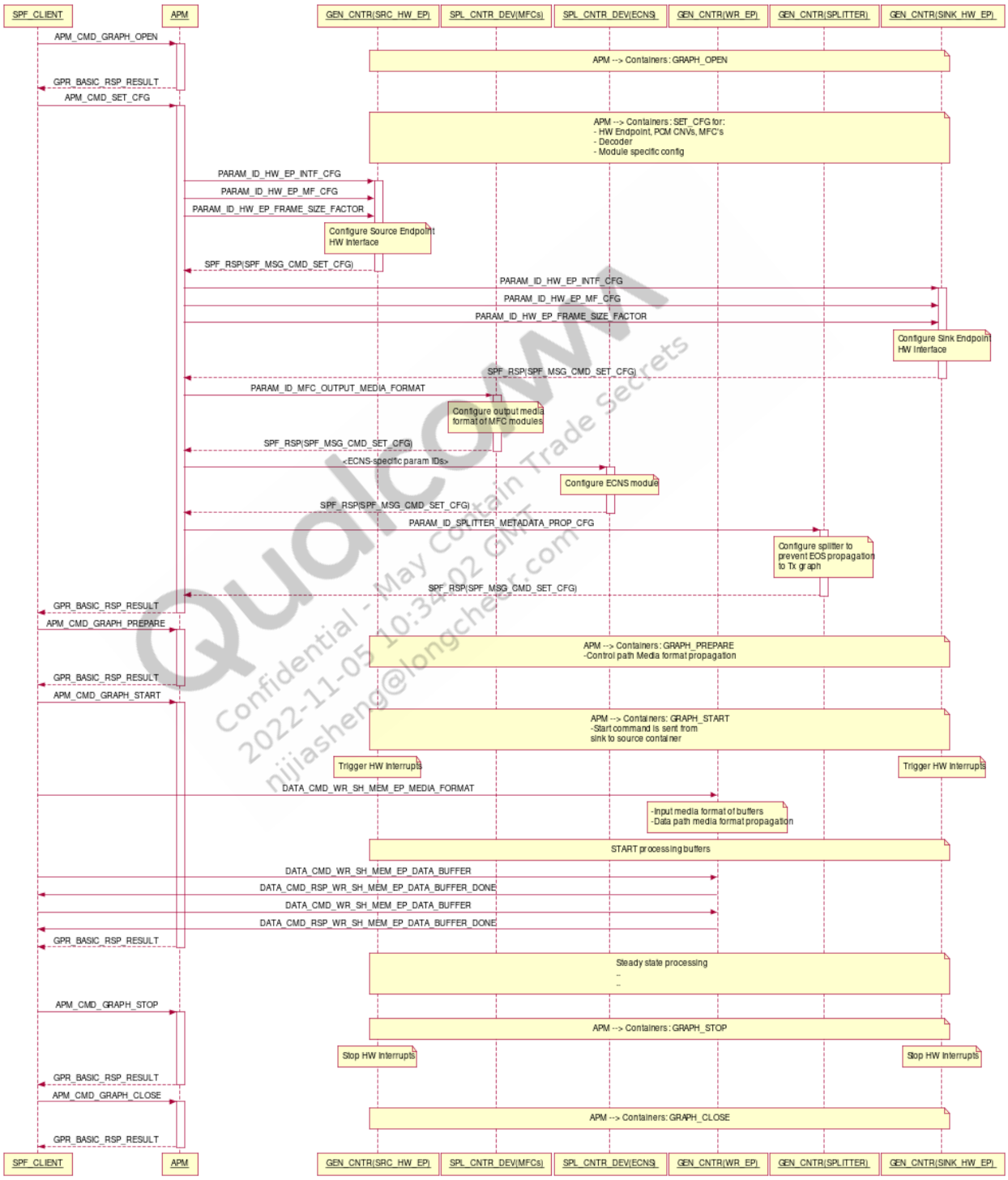
7.3.4 Karaoke

In the karaoke use case, an ECNS module is needed for mic path feedback loop. The Sample Slip module synchronizes to the speaker clock source. The Splitter module is configured to prevent sending EoS to the reference path (this could delay EoS ack to the client).



Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT nijiasheng@longcheer.com

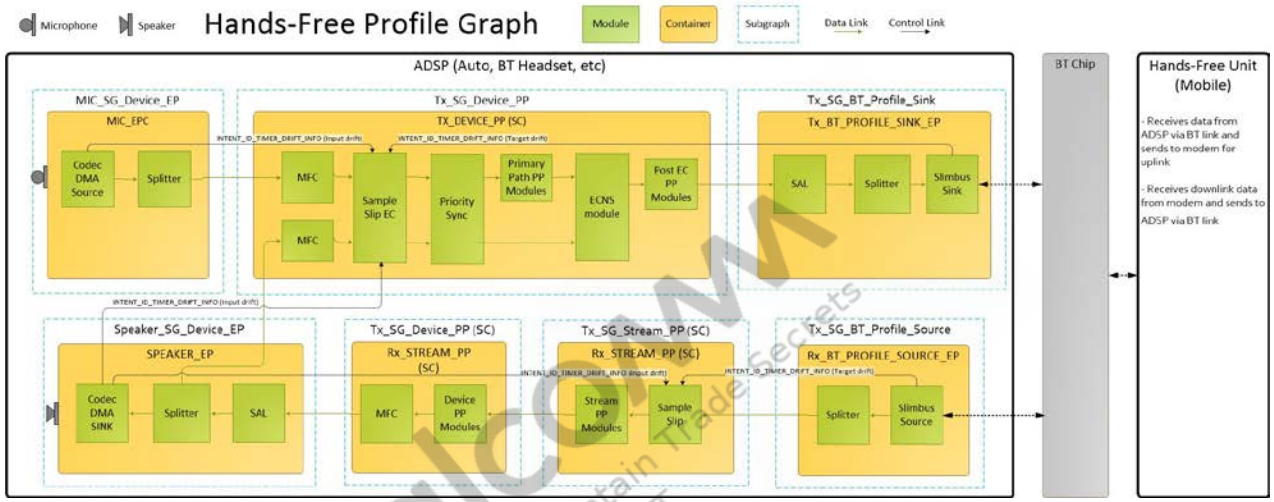
The following call flow diagram depicts the sequence of commands involved in the karaoke use case.



Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT nijiasheng@longchesr.com

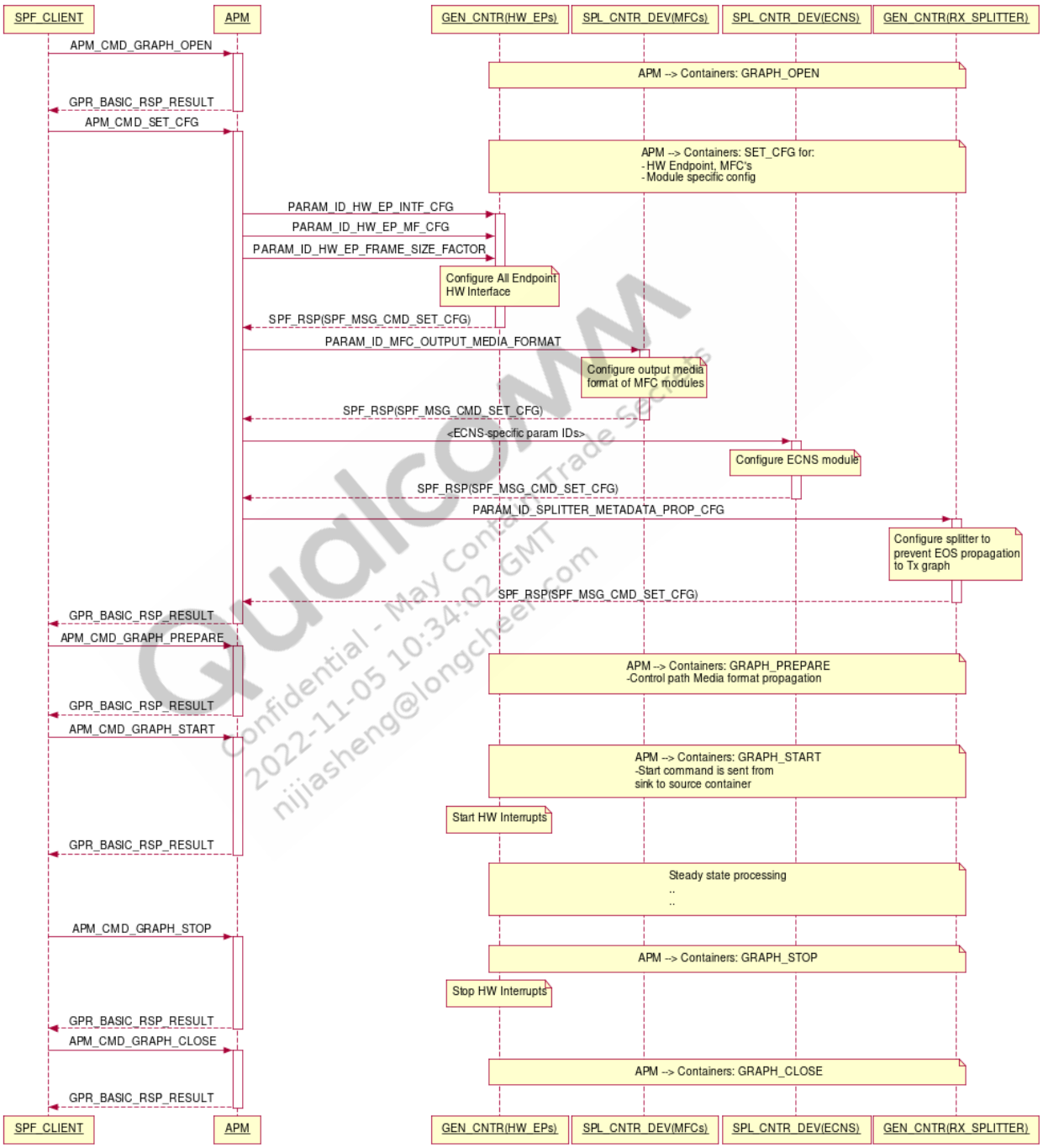
7.3.5 Hands-Free Profile (HFP)

The HFP use case involves connecting a mobile phone call to an external mic/speaker (for example, a car or headset) for a voice call. The Tx path routes mic input to a Bluetooth chip, which connects to the mobile phone. The Rx path receives Bluetooth input and routes to the speaker.



Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT nijiasheng@longcheer.com

The following call flow diagram depicts the sequence of commands involved in the HFP use case.



Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT nijiasheng@longcheer.com

8 SPF debugging

8.1 Graph-related

8.1.1 Graph management commands

The following log message information can be used to check graph management commands and responses (that is, HLOS client -> APM -> subgraphs (containers) -> APM -> HLOS client).

Item to check	Debug message string to look for (highlighted in blue)
If APM received a given command opcode from GPR	`APM GPR CB, rcvd cmd opcode [0x01001002], token:[0x0]`
APM sending commands to containers	`GRAPH_MGMT SPF_MSG[0x1001031]: cont_id:[0xE0000001]`
Whether the given command is applicable to input port/output port/control port or entire subgraph	`CNTR:20010000: cu_handle_sg_mgmt_cmd. num_ip_port_handle 1, num_op_port_handle 1, num_ctrl_port_handle 0, num_sub_graph 1` This example indicates entire subgraph was started.
Each container executing a graph management CMD: GRAPH OPEN GRAPH PREPARE GRAPH START GRAPH STOP GRAPH CLOSE GRAPH FLUSH	`GC  :20010000: CMD:START: Executing start command.` This log msg indicates generic container 2001000 began processing start command
Container completion for each command: GRAPH OPEN GRAPH PREPARE GRAPH START GRAPH STOP GRAPH CLOSE GRAPH FLUSH	`GC  :20010000: CMD:START: Done executing start command. result=0x0` This log msg indicates generic container 2001000 finished processing start command
Container to APM ack for each command	`APM RSP HDLR: Received response/event Opcode[0x01001002] , result[0x0], CONT_ID[0xE0000001]`
APM sending CONNECT command to containers as part of GRAPH OPEN	`Sent msg_opcode[0x1001035] to CONT_ID[0xE0000002]`
Container executing CONNECT command	`GC  :2000D000: CMD:CONNECT:Executing connect command, current channel mask=0x40000000`
Container result for CONNECT command	`GC  :2000D000: CMD:CONNECT:Done executing connect command, current channel mask=0x40000000. result=0x0.`

Qualcomm Confidential – May Contain Trade Secrets 2022-11-15 15:17:00 nijiasheng@longhaven.com

Item to check	Debug message string to look for (highlighted in blue)
Container to APM ack for CONNECT command	`APM RSP HDLR: Received response/event Opcode[0x1001035], result[0x0], CONT_ID[0xE0000002]`
APM to client ack	`APM RSP HDLR: GPR CMD with Opcode [0x01001002] ], SUCCESS`
Mapping of container ID in QACT to container ID in SPF	`GC :2000D000: SPF_log_parser: Created GEN_CNTR with container ID 0xE0000002, from heap_id 0` QACT ID 0xE0000002 maps to ID 2000D000 Mapping can be checked using SPF log parser

If any of these graph management commands fail, an error code will be printed while sending back ACK.
 	
For example, if GRAPH OPEN failed for container ID 0x2000D000 with error code of 0x2:
Container log print:
`2000D000: CMD:GRAPH_OPEN: Done executing graph open command. result=0x2`
 	
Result gets propagated from container to APM:
`APM RSP HDLR: Received response/event Opcode[0x01001001] , result[0x2], CONT_ID[0xE0000001]`
 	
Result gets propagated to client:
`APM RSP HDLR: GPR CMD with Opcode[0x1001001], FAILED, result: 0x2`	

8.1.2 Graph config commands

The following table provides a list of log strings for set/get configuration commands that are useful when debugging if a set/get parameter was received by APM and sent to the module as expected.

Item to check	Debug message string to look for (highlighted in blue)
APM receiving set/get cfg opcode from client through GPR	`APM GPR CB, rcvd cmd opcode`
APM parsing parameter payload for given param ID (PID) and module ID (M_IID)	`Aggregating module cfg: (M_IID, PID) = (0x401A, 0x8001024)`
APM sending set/get cfg commands to each container which has destination module	`SPF_MSG [0x100102C]: cont_id:[0xE0000008], num_mod_param[15], num_cont_param[0]` This indicates set cfg (SPF MSG set cfg opcode 0x100102C) was sent to container 0xE000008 and consists of 15 module param IDs.
Set param was sent to module and completed successfully	`10011000: set param for (param id 0x8001024, size 12) result 0` If the result is 0, it indicates the command was completed successfully, else it is an error.
Get param was sent to module and completed	`1000F000: get param for (param id 0xa00102b, size 12) result 2` If the result is 0, it indicates the command was completed successfully, else it is an error

Qualcomm Confidential – May Contain Trade Secrets 2022-11-05 19:34:02 GMT xiasheng@longsheer.com

Item to check	Debug message string to look for (highlighted in blue)
Container to APM ack for set/get cfg command	`APM RSP HDLR: Received response/event Opcode`
APM to client ack sent through GPR for set/get cfg command	`APM RSP HDLR: GPR CMD with Opcode`

8.1.3 Media format propagation

The following table shows log strings that can be used to check for media format propagation through containers and modules and help in identifying any mismatch between received media format and supported media(s) format by a module.

Item to check	Debug message string to look for (highlighted in blue)
Input media format received by each container	`1002000: SPF_log_parser: container input media format: data format 1, ch=8, SR=192000, bit_width=24, bits_per_sample=32, Q_fct=27, endianness 1, interleaving 2 (1->int,  2->deint pack, 3->deint unpack)`
Output media format raised by each container	`2001000: SPF_log_parser: container output media format: data format 1, ch=2, SR=48000, bit_width=16, bits_per_sample=32, Q_fct=27, endianness 1, interleaving 2 (1->int,  2->deint pack, 3->deint unpack)`

Output media format raised by each module	`2001000:Module 0x4003 (0x07001005): Port Id 0x1, propagating? 0, module output media format: data_format 1, fmt_id 0x9001000`
	
	`module output media format: data format 1, ch=2, SR=48000, bit_width=16, bits_per_sample=16, Q_fct=15, endianness 1, interleaving 2 (1->int,  2->deint pack, 3->deint unpack)`
	
	Here 2001000 is the container ID, 0x4003 is the MIID, and 0x07001005 is the module ID.

8.2 Module-related

8.2.1 Hardware EP modules

The following log information is used to check if EP modules are started successfully and all required configuration parameters are set properly.

Item to check	Debug message string to look for (highlighted in Blue)
EP is started successfully	`Codec DMA interface port intf 1 successfully opened for lpaif_type 2 with direction 1 (CODEC_DMA_SINK = 1, CODEC_DMA_SOURCE = 2)`
EP hardware media format configuration set	`CODEC_DMA_Drv: Received hw media format config: sample rate = 48000, bit width = 16, num channels = 1, data format = 1`
EP interface configuration set	`CODEC_DMA_Drv: Received interface config: codec_dma_type = 2, intf_indx = 1, active_channels_mask = 3`

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Item to check	Debug message string to look for (highlighted in Blue)
If EP mandatory configuration is not received or failed	• `CAPI_CODEC_DMA: Driver set intf cfg failed` • `CAPI_CODEC_DMA: Driver set ep mf failed` • `CAPI_CODEC_DMA: Cannot start, configuration not validated- Intf cfg received: %d Media fmt received: %d`
Underruns from within sink endpoint module. It could indicate: ▪ Input media format not received ▪ Input buffer not received ▪ Input buffer size less than expected. Note: ▪ Underrun prints do not always imply an issue like the messages printed at the start of a use case before input media format or buffer are received ▪ When an EP underruns, it pushes zeros to the output	`CAPI_CODEC_DMA: underrun`
To check similar prints for other hardware EP modules, search for capi_<module_name> where module name can be I2S/TDM/display_port/USB/SLIMbus	

8.2.2 WR SH Mem EP

The following logging information can be used to check if write data buffers are being sent from the HLOS client to the WR SH Mem EP module and SPF is processing and returning these write buffers.

This sequence is useful to detect stalls in the data flow due to the HLOS client not sending buffers or when processing in SPF stalls leading to buffers not being sent back.

Item to check	Debug message string to look for (highlighted in blue)
Receiving WR buffers from HLOS client	`GPR callback from src 0x2010 to dst 0x4001, opcode 0x4001000, token 0x1`
Return WR buffers to HLOS client (if there is a stall in ADSP, buffers will not be returned on time).	`0x4001: status 0x0, DATA_CMD_RSP_WR_SH_MEM_EP_DATA_BUFFER_DONE 0x5001000, token 0x1`

Qualcomm Confidential - May Contain Trade Secrets 2022-11-03 10:34:02 GMT nijia-hong@lonestar.com

8.2.3 RD SH Mem EP

The following logging information can be used to check if read data buffers are being sent from the HLOS client to the RD SH Mem EP module and if SPF is processing and returning these read buffers.

This sequence is useful to detect stalls in the data flow due to the HLOS client not sending buffers or when processing in SPF stalls leading to buffers not being sent back.

Item to check	Debug message string to look for (highlighted in blue)
Receiving RD buffers from client	`GPR callback from src 0x600B to dst 0x6001, opcode 0x4001003, token 0x1`
Return RD buffers (if there is a stall in ADSP, these buffers will now be returned on time)	`0x6001: status 0x0, DATA_CMD_RSP_RD_SH_MEM_EP_DATA_BUFFER_DONE [0xb0909948,40]`

8.2.4 Data Logging

Data logging modules can be added in the use case graph to check audio data at each of those locations to figure out at what point audio data got noisy/muted. The following table provides information on details contained in an audio data log name.

Example log name: 02-26.13-53.isf.0x1586.pcm.0x30020041.0x2.0x2.rx	
02-26.13-53.isf	Name of the QXDM file
0x1586	Log code: QXDM ID for logging
pcm	Data format
0x30020041	Container log ID
0x2	Logging tap point ID
0x2	Channel
Rx	Direction

The log code does not necessarily correspond to a fixed position of the data logging module in the graph (for example, 1586 does not always mean logging in EP, even though that is usually recommended).

8.2.5 MFC

The following log string information can be used to check information specific to MFC modules.

Item to check	Debug message string to look for (highlighted in blue)
MFC output media format received through set configuration	`MFC_PCM_CNV:[40BB] [3] PARAM_ID_MFC_OUTPUT_MEDIA_FORMAT SR: 48000 CH: 2 BW: 16`

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT lijiasheng@lobshare.com

8.2.6 Placeholder

The following logging information can be used to check real module ID information and reset received by placeholder modules.

Item to check	Debug message string to look for (highlighted in blue)
Placeholder module received real module ID	`GC :20010000: SPF_log_parser: Placeholder 0x00004082 received Real Module ID 0x700103b`
Placeholder module received reset	`Module 0x%lX: placeholder module reset`

8.2.7 SPR

Item to check	Debug message string to look for (highlighted in blue)
Session time query print	`capi_spr: 00004164: get: session_time: [session time msw, session time lsw] = [0, 0] : [absolute time msw, absolute time lsw] = [0, 0]:[time stamp msw, time stamp lsw] = [0, 0]:Flags 0x0`
SPR timer started	`capi_spr: 00004164: started timer, absolute time 792380415 us`

8.2.8 Soft Pause

Item to check	Debug message string to look for (highlighted in blue)
Pause Module received START PAUSE	`capi_soft_pause: Set param PARAM_ID_SOFT_PAUSE_START received`
Pause module received RESUME	`capi_soft_pause: Set param PARAM_ID_SOFT_PAUSE_RESUME received`
Pause module received ramp parameters	`capi_soft_pause: Set param PARAM_ID_SOFT_PAUSE_RAMP_PARAMETERS received with ramp params-> down: p:40 s:1000 r:1 up: p:40 s:1000 r:1`

Qualcomm Confidential – May Contain Trade Secrets 2022-11-08 10:24:02 GMT nijianhe@qualcomm.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.3 Common issues

The following subsections provide debug information for common issues with audio playback/record use cases. For any kind of issue, it is suggested to first check for the `error` string in QXDM log.

8.3.1 Output is muted

The following table lists possible issues that may have resulted in audio playback/record mute and the corresponding log messages to check for in the QXDM log.

Possible Issues	Log messages to look for
Media format mismatch ▪ Input media format received by module in the data path does not match media format configuration provided to module. ▪ Input media format received is not supported by module	▪ Verify media format propagation through the modules by searching for the `module output media format` print in QXDM logs or through the SPF log parser. ▪ Configure the PCM Converter module or MFC module in the path before the module to ensure media format matches
Underruns ▪ Underruns are not the cause of the issue but indicate the possible issue ▪ Underruns occur on signal triggered modules/containers ▪ Underrun on a module/container port indicates data is not available on that port on time (that is, upstream is sending data slower, or upstream data is stalled).	Check for the `underrun` print in the QXDM log. If output is muted, continuous underruns might be occurring on an input port of a module/container. SPR module: ▪ Check if client is sending buffers using prints in Section 8.2.2 ▪ When a topology is paused, underruns are expected on SPR which is not an error condition. Hardware EP module: ▪ Check if it is one of the factors covered in Section 8.2.1
An overrun on a module port indicates data is available but is not being consumed on time (that is, downstream is not consuming data fast enough or downstream is stalled)	Check for the `overrun` print in QXDM log. If output is muted, overruns might be occurring on every trigger on the output port of modules/containers due to a stall in downstream processing.
Data drops – Data might be getting dropped at a module depending on module behavior	Search for the debug string `Dropping`

Qualcomm Confidential – May Contain Trade Secrets 2022-11-05 10:34:02 GMT xiaosheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.3.2 Output is noisy

The following table lists possible issues that may have resulted in noisy output and the corresponding log messages to check for in the QXDM log.

Possible Issues	Log messages to look for
Underruns ▪ Underruns are not the cause of the issue but indicate the possible issue ▪ Underruns occur on signal triggered modules/containers ▪ Underrun on a module/container port indicates data is not available on that port on time (that is, upstream is sending data slower, or upstream data is stalled).	Check for the **`underrun`** print in the QXDM log. If output is noisy, underruns might be occurring every few triggers on an input port of a module/container. ▪ SPR module – Check if the client is sending buffers using prints in Section 8.2.2 ▪ Upstream not processing on time due to lower thread priority, not voting for enough mips, etc.
An overrun on a module port indicates data is available but is not being consumed on time (that is, downstream is not consuming data fast enough or downstream is stalled)	Check for the **`overrun`** print in the QXDM log. If output is muted, overruns might be occurring on every trigger on an output port of modules/containers due to a stall in downstream processing.
Data drops – Data might be getting dropped at a module depending on module behavior	Search for the debug string **`Dropping`**

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 10:34:02 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

A References

A.1 Related documents

Title	Number
Qualcomm Technologies, Inc.	
AudioReach Signal Processing Framework Technical Overview	80-VN500-16
GPR Packet Sniff Tool	80-VN500-12
CAPI Custom Module Integration AudioReach SPF	80-VN500-17
AudioReach™ SPF Common Audio Processor Interface (CAPI)	80-VN500-6

A.2 Acronyms and terms

Acronym or term	Definition
ACDB	Audio Calibration Database
ADSP	Application Digital Signal Processor
HLOS	High-Level Operating System
LPASS	Low Power Audio Subsystem
MID	Module ID
MIID	Module Instance ID
GPR	Graph Packet Router
APM	Audio Processing Manager
EP	End Point
SPF	Signal Processing Framework
WR	Write
SSMD	Single Stream Multiple Devices
MSMD	Multiple Streams Multiple Devices

Qualcomm Confidential – May Contain Trade Secrets 2022-10-05 10:24:02 GMT Niasher@dlorcheser.com